

Kofi Wilkinson

**Next Generation CHERI Tag
Cotnroller**

Computer Science Tripos – Part II

Homerton College

May 10, 2024

Proforma

Name: **Kofi Wilkinson**
College: **Homerton College**
Project Title: **Next Generation CHERI Tag Controller**
Examination: **Computer Science Tripos – Part II, June 2024**
Word Count: **(Intro+Prep+Impl = about 11,000) ¹**
Project Originator: **Dr Jonathan Woodruff**
Supervisor: **Dr Jonathan Woodruff**

Original Aims of the Project

To develop a framework in which a computer architect can implement and evaluate the algorithms of a memory access controller for use in the microarchitecture of a tagged memory architecture. To also use the framework to evaluate two existing tag controller implementations.

Work Completed

I successfully developed the framework, and used it to evaluate two existing tag controller implementations. I also developed an additional visualisation tool allowing insight into the workings of the tag controllers to be gained visually. I then used these tools to develop a novel tag controller of my own design, which I evaluated and compared to the existing ones.

Special Difficulties

None.

¹This word count was computed by `texcount main.tex -inc`

Declaration

I, Kofi Wilkinson of Homerton College, being a candidate for Part II of the Computer Science Tripos, hereby declare that this dissertation and the work described in it are my own work, unaided except as may be specified below, and that the dissertation does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this dissertation I did not use text from AI-assisted platforms generating natural language answers to user queries, including but not limited to Chat-GPT. I am content for my dissertation to be made available to the students and staff of the University.

Signed

A handwritten signature in black ink, appearing to read "K. Wilkinson", with a long horizontal stroke extending to the left.

Date May 8, 2024

Contents

1	Introduction	13
1.1	Tagged Memory Architectures	13
1.2	Question 1 – Existing Mitigations	13
1.3	Question 2 – Potential Areas for Improvement	14
1.4	My Project	14
2	Preparation	15
2.1	Background Material	15
2.1.1	Tagged Memory Architectures	15
2.1.2	CHERI	16
2.1.3	A Basic Scheme	17
2.1.4	The Importance of a Tag Cache	19
2.1.5	<i>Efficient Tagged Memory</i> ’s Hierarchical Scheme	20
2.1.6	The Morello Project Scheme	24
2.2	Three-Stage Simulation	25
2.2.1	Workload Execution – LLC Miss Simulation	26
2.2.2	A Tag Controller Simulator	26
2.2.3	ChampSim – An Appropriate Cache Simulator	26
2.2.4	Visualisation Tool	26
2.2.5	Requirements Analysis	27
2.3	Software Engineering Practices	27
2.3.1	Software Engineering Methodology	27
2.3.2	Build system	27
2.3.3	File Backups and Version Control	28
2.3.4	Software Licences	28
2.4	Starting Point	28
3	Implementation	29
3.1	Simulator	29
3.1.1	Simulator Framework	30
3.1.2	Trace Decoder	31
3.1.3	Simulated Memory	36
3.1.4	Tag Controller	38
3.1.4.1	Flat Table Controllers	39

3.1.4.2	Basic Scheme	40
3.1.4.3	<i>Efficient Tagged Memory's</i> Hierarchical Scheme	41
3.1.4.4	The Morello Project Scheme	43
3.2	Logging	46
3.2.1	Logging Tag Table State	46
3.2.2	Adding Logging to ChampSim for Tag Cache State	47
3.3	Verification	47
3.3.1	Microbenchmarks – Unit Tests	47
3.3.2	Macrobenchmark – Trace Verifier	47
3.3.3	Macrobenchmark – Cross-Verifier	48
3.4	Visualisation Tool	49
3.5	Phoenix – A Novel Scheme	51
3.5.1	The Motivation	51
3.5.2	The Intuition	51
3.5.2.1	Reference Counters	51
3.5.2.2	The Superroot Table	53
3.5.2.3	The Tag Working Set	55
3.5.3	The Algorithm	57
3.5.4	The Implementation	61
3.6	Repository Overview	62
4	Evaluation	63
4.1	DRAM Access Latency	63
4.1.1	Flat Table Controllers	63
4.1.1.1	Performance	64
4.1.1.2	Varying Capacity	65
4.1.1.3	Varying Ways	66
4.1.1.4	Adding a Tag Cache Prefetcher	67
4.1.2	Hierarchical Schemes	68
4.1.2.1	Performance	68
4.1.2.2	Varying Capacity	71
4.1.2.3	Varying Ways	72
4.1.2.4	Adding a Tag Cache Prefetcher	73
4.2	DRAM Space Consumption	74
4.2.1	Theoretical Limits of Tag Table Compression	74
4.2.2	Dynamic DRAM Allocation	74
4.3	Evaluation of the Simulator as a Tool	75
5	Conclusions	77
5.1	Project Successes	77
5.2	Future Work	77
5.3	Reflection and Lessons Learned	77
	Bibliography	78

A Dummy	81
B Project Proposal	83

List of Figures

2.1	How the two parts of a tagged value are stored in physical memory.	15
2.2	Step-by-step memory access translation.	16
2.3	How the parts of the memory system interface.	16
2.4	A CHERI capability.	17
2.5	The basic algorithm for reading tags. Memory access operations are blue. .	17
2.6	The basic algorithm for writing tags. Operations internal to the tag controller are grey.	18
2.7	Where the tag cache sits in the memory system.	20
2.8	<i>Efficient Tagged Memory</i> 's two-level hierarchical lookup table.	21
2.9	The ETM algorithm for reading tags.	21
2.10	The ETM algorithm for writing tags.	22
2.11	The physical layout of the hierarchical table.	23
2.12	The layout of the two tag caches in the Morello memory system.	25
2.13	A black-box diagram explaining the three stage simulation.	25
3.1	An explanation of how the three main parts of the tag controller simulator interact with the inputs and outputs, and with each other.	29
3.2	The layout of an <code>llcMiss</code> struct.	32
3.3	An illustration of how the LLC miss files were generated by the CHERI processor simulator.	33
3.4	An example of how only partial information is known about the state of tags in the table.	34
3.5	The rubric for assuming the value of an unknown tag.	35
3.6	The UML diagram detailing the inheritance of the tag controller classes. .	39
3.7	The Morello Project algorithm for reading tags. Pseudo-memory access operations are purple.	45
3.8	The Morello Project algorithm for writing tags.	46
3.9	A snapshot of the tag table when running the <i>Efficient Tagged Memory</i> scheme.	50
3.10	A closer look at the root table.	50
3.11	A closer look at the leaf table.	51
3.12	An illustration of the table of tag counters.	52
3.13	The Phoenix scheme's three-level hierarchical lookup table.	54
3.14	The physical layout of the hierarchical table in the Phoenix scheme.	55

3.15	The Phoenix algorithm for reading tags. Tag working set operations are in red.	58
3.16	The Phoenix algorithm for writing tags. DRAM allocator operations are orange.	60
3.17	A visualisation of the implementation of the tag working set, and how the replacement policies choose their victims.	61
4.1	Tag cache access frequencies of the flat table controllers on various workloads.	64
4.2	Tag cache miss frequencies of the flat table controllers on various workloads.	64
4.3	How tag cache size affects miss frequencies of the flat table controllers. . .	65
4.4	How the number of ways in the tag cache affects miss frequencies of the flat table controllers.	66
4.5	How adding a prefetcher to the tag cache affects miss frequencies of the flat table controllers.	67
4.6	Tag cache access frequencies of the hierarchical table controllers on various workloads.	68
4.7	Tag cache miss frequencies of the hierarchical table controllers on various workloads.	69
4.8	The worst-case number of dependent memory accesses of the hierarchical table tag controller algorithms.	70
4.9	Best- and worst-case DRAM access latency overhead for the hierarchical table tag controllers.	70
4.10	How tag cache size affects miss frequencies of the hierarchical table controllers.	71
4.11	How the number of ways in the tag cache affects miss frequencies of the hierarchical table controllers.	72
4.12	How adding a prefetcher to the tag cache affects miss frequencies of the hierarchical table controllers.	73
4.13	Various compression algorithms applied to the tag tables of various workloads.	74
4.14	The space consumption of the tag table when using the Phoenix scheme's on-demand DRAM allocation.	75

Acknowledgements

—

Chapter 1

Introduction

1.1 Tagged Memory Architectures

The paper *Efficient Tagged Memory* [1] describes how tagged memory architectures have been explored in academia and employed in industry many times with various different focuses throughout the history of computer design. Through both academic rigour and market testing they have been proven to provide desirable system-wide security properties such as hardware capabilities [2], pointer integrity [3], watchpoints [4], and information-flow tracking [5].

There are however associated costs that render widespread adoption of tagged memory architectures currently infeasible. With a simplistic scheme, the number of memory accesses that workloads must perform doubles due to the CPU now having to access the tag of the value it writes, as well as the value itself, which brings an unacceptably large time overhead. Also, the introduction of the table in which tags are stored comes with a constant memory consumption cost, which is particularly disadvantageous within the server-side compute industry, where every byte of DRAM is precious.

1.2 Question 1 – Existing Mitigations

Efforts have been and are currently being made to mitigate the overheads of tagged memory. Tag caches are often used in practice to hold tag data closer to the CPU for faster access times. The main area in which mitigation of the memory access latency overhead is found is in increasing the ‘cacheability’ of the tag table in DRAM. That is, by increasing the amount of information about the tags that we store in the fast-access tag cache, we reduce the average latency overhead that is introduced by tagged memory’s new requirement of accessing tag data. An effort of this kind, described in *Efficient Tagged Memory*, has been shown to reduce the common-case DRAM traffic overhead to “well below 1%”.

The first question that my dissertation concerns itself with is as follows:

“Question 1: How do existing implementations aim to mitigate the overheads that tagged memory introduces, and how effective are they at doing so?”

1.3 Question 2 – Potential Areas for Improvement

There has been little documented research on attempts to address the memory consumption cost that renders tagged memory a non-starter in server applications. A potential solution to this problem might lie in the use of a hypervisor to dynamically allocate DRAM to guest OSes. Suppose that the tagged memory system is one which runs a hypervisor – a common setup for servers. Suppose further that the hypervisor has a dynamic DRAM allocator that could offer free physical address regions to its guest operating systems as and when they require it. A novel system involving a dynamically-resizing tag table that can be compressed and decompressed at runtime would allow the guest OSes to reserve and free regions of DRAM that they use for storing their tags as they execute, meaning that each OS will consume large amounts of extra space only when necessary, as opposed to all of them reserving at boot-time the maximum that they might require during any part of their execution. By increasing the ‘compressibility’ of the tag table, we can keep the size of the tag table in each guest OS instance closer to a theoretical minimum, reducing the additional memory consumed by the table.

The second question that my dissertation concerns itself with is as follows:

“Question 2: How can we further reduce the overheads that tagged memory introduces?”

1.4 My Project

The utility of tagged memory architectures motivates the exploration and investigation of existing tag controller schemes in search for opportunities to improve upon them – to increase the ‘cacheability’ and ‘compressibility’ of the tag table. Previously, frameworks for implementing tag controllers have been developed, but they were either proprietary, heavily undocumented, or not intended for public use. However, it is from research performed using these frameworks where the majority of innovation in tagged memory architecture implementations has arisen.

For my project, I aim to develop a framework in which an architect can implement tag controllers of their own design, and evaluate them quantitatively. This would allow rapid iteration of designs, and thus would help to foster innovation. I will use the framework to evaluate a handful of existing tag controller implementations, addressing Question 1. I also aim to develop a tag controller of my own that builds on ideas from existing implementations, proving the utility of the framework both as a tool for understanding the strengths of current controllers and for evaluating new ones. By developing both the framework and my novel tag controller, I hope to be able to provide a contribution to the field of research in this area, and to participate in answering Question 2.

Chapter 2

Preparation

This chapter explains the relevant background material required to undertake the project and to understand the dissertation. It also details the major pieces of my project, how they fit together, and what was required to produce a successful implementation.

2.1 Background Material

2.1.1 Tagged Memory Architectures

In a tagged memory architecture N -bit values in memory are notionally ‘extended’ with M -bit tags. Doing so can provide desirable security properties like those mentioned in the introduction. *Efficient Tagged Memory* [1] describes how a one-bit tag is sufficient to implement all of the security properties that tagged memory architectures can afford. The extension of values with tags might be implemented in one of several ways, but section III of the paper details how storing all of the tag bits in a single table at the bottom of DRAM is the most practical for single-bit tag (SBT) architectures; this is also the most used technique in practice.

Typically, these architectures mandate allocation of a static region of DRAM at boot-time to act as the tag table. This results in a constant physical memory consumption cost that is proportional to the size of the (taggable) address space.

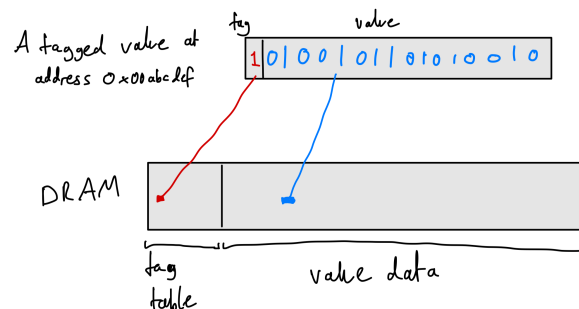


Figure 2.1: How the two parts of a tagged value are stored in physical memory.

To implement such a scheme in hardware, we use a tag controller. This unit is responsible for intercepting memory access requests and replacing them with appropriate requests that will result in the retrieval of both the value and its tag.

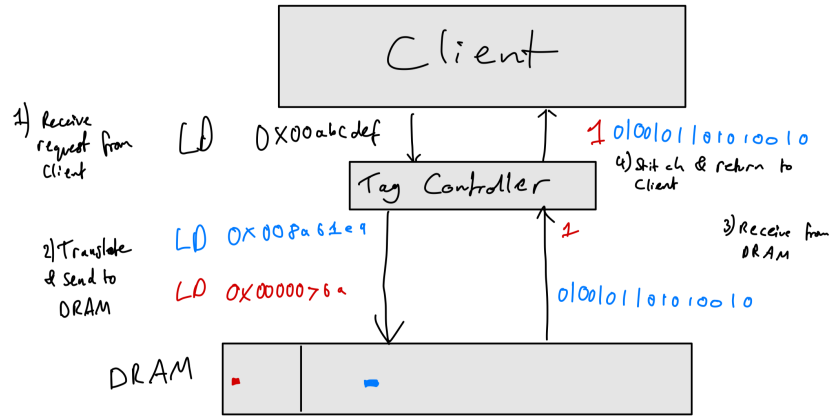


Figure 2.2: Step-by-step memory access translation.

My project focuses on merged-cache hierarchy architectures. In these, values in the cache are stored unified with their tag – that is, the data caches contain tags as well as values. Section IV of *Efficient Tagged Memory* explains how the alternative, split-cache hierarchy architectures, are ‘unnecessary’, and result in a less efficient and less practical implementation. With merged-cache hierarchies, the tag controller only needs to intercept last-level cache (LLC) misses. The tag controller therefore interfaces with the LLC and DRAM.

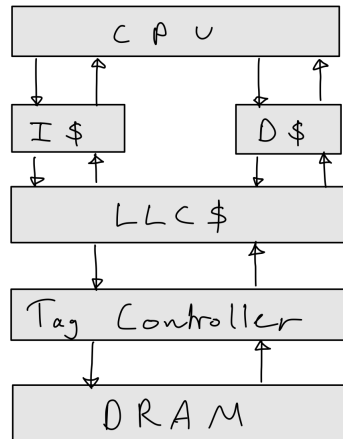


Figure 2.3: How the parts of the memory system interface.

2.1.2 CHERI

My project focuses on the CHERI architecture model. The model defines a single-bit tagged memory architecture, where 64-bit pointers can be replaced with 128-bit ‘capabilities’ that are also given a 1-bit tag, which allows hardware-enforced security invariants to be guaranteed about these special values, and thus also about executions of programs

that use them. There is more information about the CHERI model on the website for the Department of Computer Science and Technology at the University of Cambridge [6].

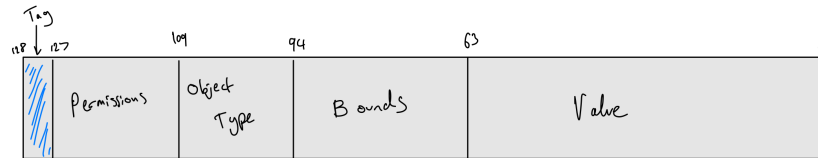


Figure 2.4: A CHERI capability.

My project focuses particularly on the CHERI architecture as it is a speciality of the university and of my supervisor, and it is also the tagged memory architecture that I am most familiar with, as it was briefly introduced to us in IB, and I also spent some time working with it over the previous summer during my internship as a member of Arm’s Morello Kernel Team. Much of my work and findings however will be transferable to other tagged memory architectures.

2.1.3 A Basic Scheme

The most basic scheme has a single flat table at the bottom* of DRAM that contains all of the tags, and nothing more. This table contains one tag bit per 128 bits of regular memory (as a CHERI capability is 128-bits plus a 1-bit tag). The table therefore consumes the bottom 129th of DRAM. The layout of DRAM is as depicted in figure 2.1.

* Note that some implementations instead put the table at the top of DRAM. This is functionally equivalent, though of course the addresses used will differ.

When the tag controller retrieves tags by reading from the tag table, it uses the following algorithm:

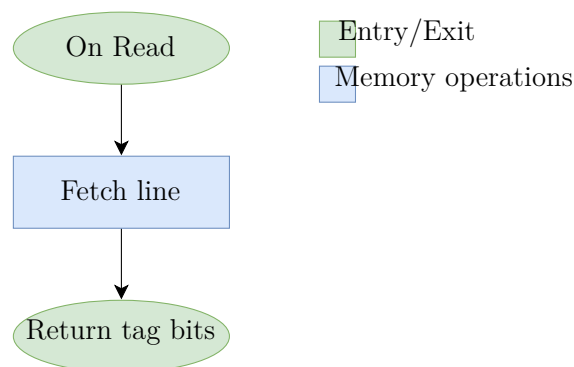


Figure 2.5: The basic algorithm for reading tags. Memory access operations are blue.

When the tag controller updates the state of tags by writing to the tag table, it uses the following algorithm:

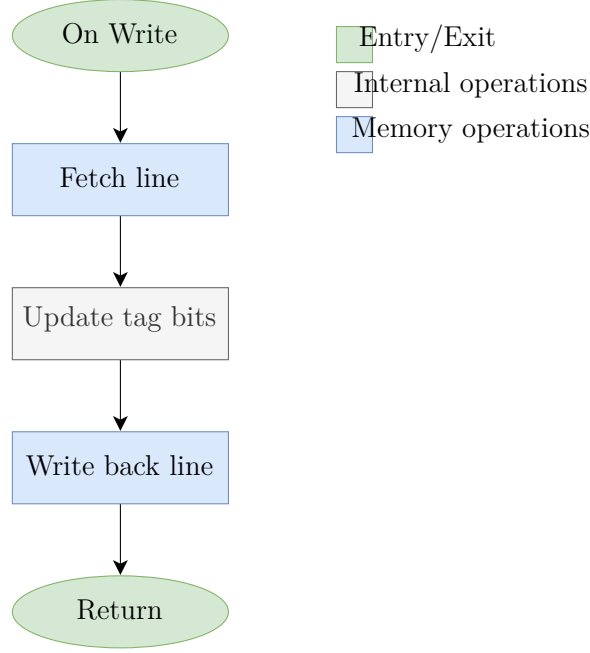


Figure 2.6: The basic algorithm for writing tags. Operations internal to the tag controller are grey.

As far as the processor and the cache systems are concerned, regular memory still begins at physical address $0x0$. The fact that the controller is mandating that the base of the *tag table* is at $0x0$, rather than the base of physical memory being here, is abstracted away by the controller’s interface. For clarity, I will refer to the addresses that the processor and caches see as ‘logical physical addresses’, and those that the tag controller emits and DRAM sees as ‘actual physical addresses’. These are both distinct from virtual addresses, which are not relevant here.

Also, note that everything here is done in terms of cachelines, as the tag controller’s client is a cache – the tag controller receives logical base addresses of cachelines from the LLC, rather than those of individual values, and it must return entire cachelines to the LLC.

So, suppose the last-level cache misses the cacheline with logical base address a . It sends a request to the tag controller to retrieve the cacheline that lives at logical address a , and also its corresponding tags. The controller then uses the following formulae to calculate which actual physical addresses it needs to access in DRAM:

$$\text{Actual Value Cacheline Address} = a + \frac{\text{Physical Memory Size}}{128} \quad (2.1)$$

$$\text{Actual Tag Cacheline Address} = \left\lfloor \frac{a}{128} \right\rfloor_{64} \quad (2.2)$$

Where the notation $\lfloor x \rfloor_y$ denotes the nearest multiple of y that is less than or equal to x – i.e. rounding x down to the nearest multiple of y .

For the actual base address of the value cacheline, we must take into account the offset of the value portion of DRAM. The value portion begins immediately after the end of the tag table. The tag table has size $\frac{\text{Physical Memory Size}}{128}$, thus we add this value onto the logical

address of the line, a , to get the actual physical address of the line.

The actual base address of the tag cacheline will index into the tag table. In this scheme, the tags are arranged in the same order as the 128-bit values are themselves in DRAM. That is, the first bit of the tag table is the tag of the first 128-bit value in DRAM. The second bit is that of the second 128-bit value, and so on. As we assume each cacheline is the industry standard of 64 bytes, and that each tag corresponds to 128 bits (16 bytes), there are 4 tags per value cacheline. Thus, **one cacheline of tags from the tag table** holds the tags for **128 value cachelines** (see equation 2.3) By dividing a by 128 we get the location in the tag table that corresponds to the location of the value in the value portion of DRAM. We then must calculate the base address of the tag cacheline which this location lies in, which is done by rounding down to the nearest multiple of 64, as the tag cachelines are 64 bytes.

$$\frac{512 \text{ tags per tag cacheline}}{4 \text{ tags per value cacheline}} = 128 \text{ value cachelines covered by one tag cacheline} \quad (2.3)$$

2.1.4 The Importance of a Tag Cache

On retrieving a cacheline, the tag controller must make two memory accesses; one to retrieve the value cacheline, and another to retrieve the tag cacheline, from which the appropriate tags are extracted and merged with the values, before returning the result to its LLC client. The fact that we now have to make several memory accesses when retrieving a single cacheline is why we observe an overhead in lookup latency. Typically, the access to the value cacheline and the access to the tag cacheline can be pipelined, but this still increases the latency by a factor of about 1.1. On the other hand, the algorithm for writing contains two accesses with a data dependency, and thus these cannot be pipelined, so the latency of LLC write misses increases by a factor of 2.

Without adding another level of data cache, we cannot avoid accessing the value cacheline in DRAM. What we can do though is introduce a new cache at this level of the memory hierarchy, just below the LLC and alongside the tag controller, that stores cachelines from *the DRAM tag table*. This is known as the tag cache.

When accessing memory with the presence of a tag cache, we first check to see if the tag cacheline we want is in the tag cache. This request can be performed in parallel to the DRAM lookup of the value cacheline. If the tag cache happens to hit, then we avoid the second DRAM access entirely, and eliminate the overhead. Naturally, this motivates efforts to maximise the hit rate of the tag cache.

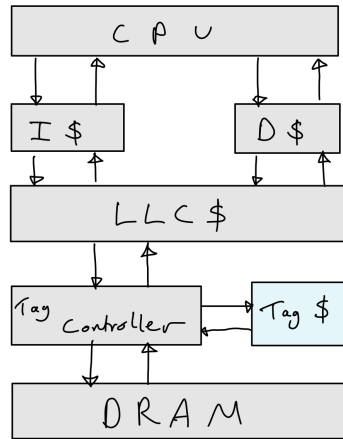


Figure 2.7: Where the tag cache sits in the memory system.

2.1.5 *Efficient Tagged Memory's* Hierarchical Scheme

One of the tag controller schemes that my project focuses on is the scheme described in *Efficient Tagged Memory*. This is in part because of its simplicity, but also because my supervisor contributed to the paper; his familiarity with the scheme allowed him to guide me easily and effectively where necessary.

One way to increase the hit rate of the tag cache is to store more information about the tag table within it. This can be done by simply increasing its size. However, there is an alternative – by adjusting the scheme, we can increase the ‘cacheability’ of the tag table. That is, we can fit more information about the tag table in the same size tag cache. Section VI of *Efficient Tagged Memory* details a tag table scheme that does just this.

This scheme uses a two-tiered hierarchical tag table. The ‘leaf’ table is identical to the table described in the basic scheme, with one tag bit per 128-bit value. The ‘root’ table contains bits that hold information about *the state of the tags in the leaf table*.

One bit in the root table corresponds to 512 bits in the leaf table – an entire cacheline of tags. The root bit is a 0 if all 512 bits in the corresponding leaf cacheline are 0, otherwise it is a 1.

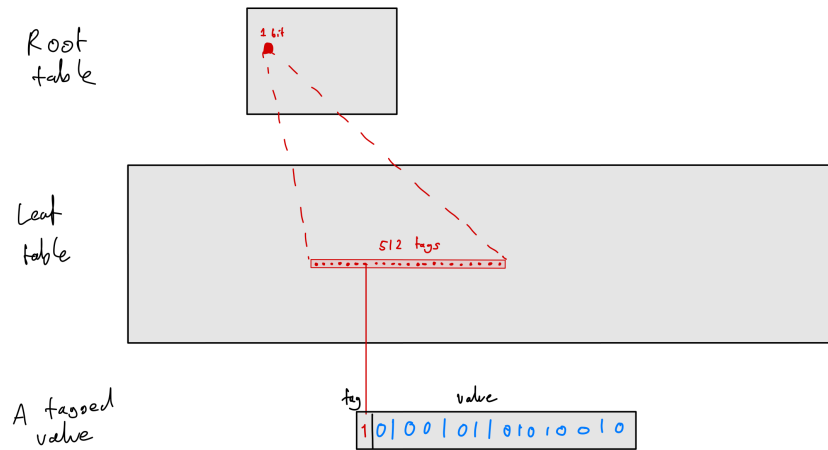


Figure 2.8: Efficient Tagged Memory's two-level hierarchical lookup table.

The tag controller has a new algorithm for both reading and writing the state of tags in the table. The algorithm for reading is as follows:

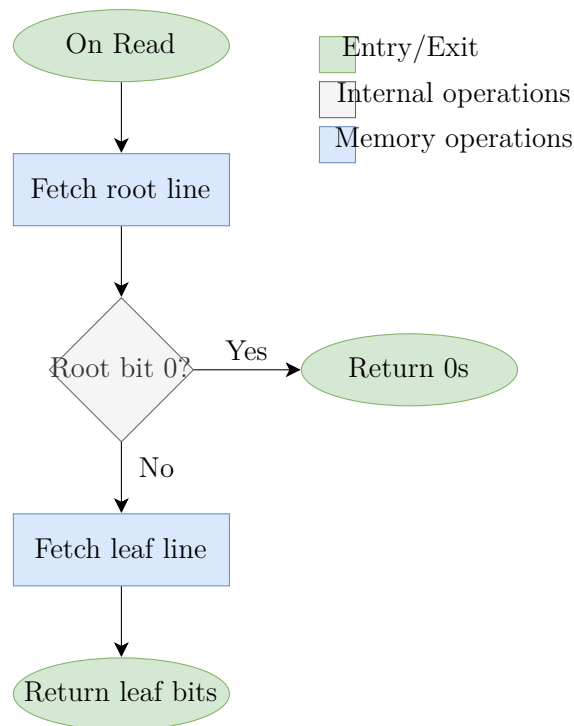


Figure 2.9: The ETM algorithm for reading tags.

The algorithm for writing is as follows:

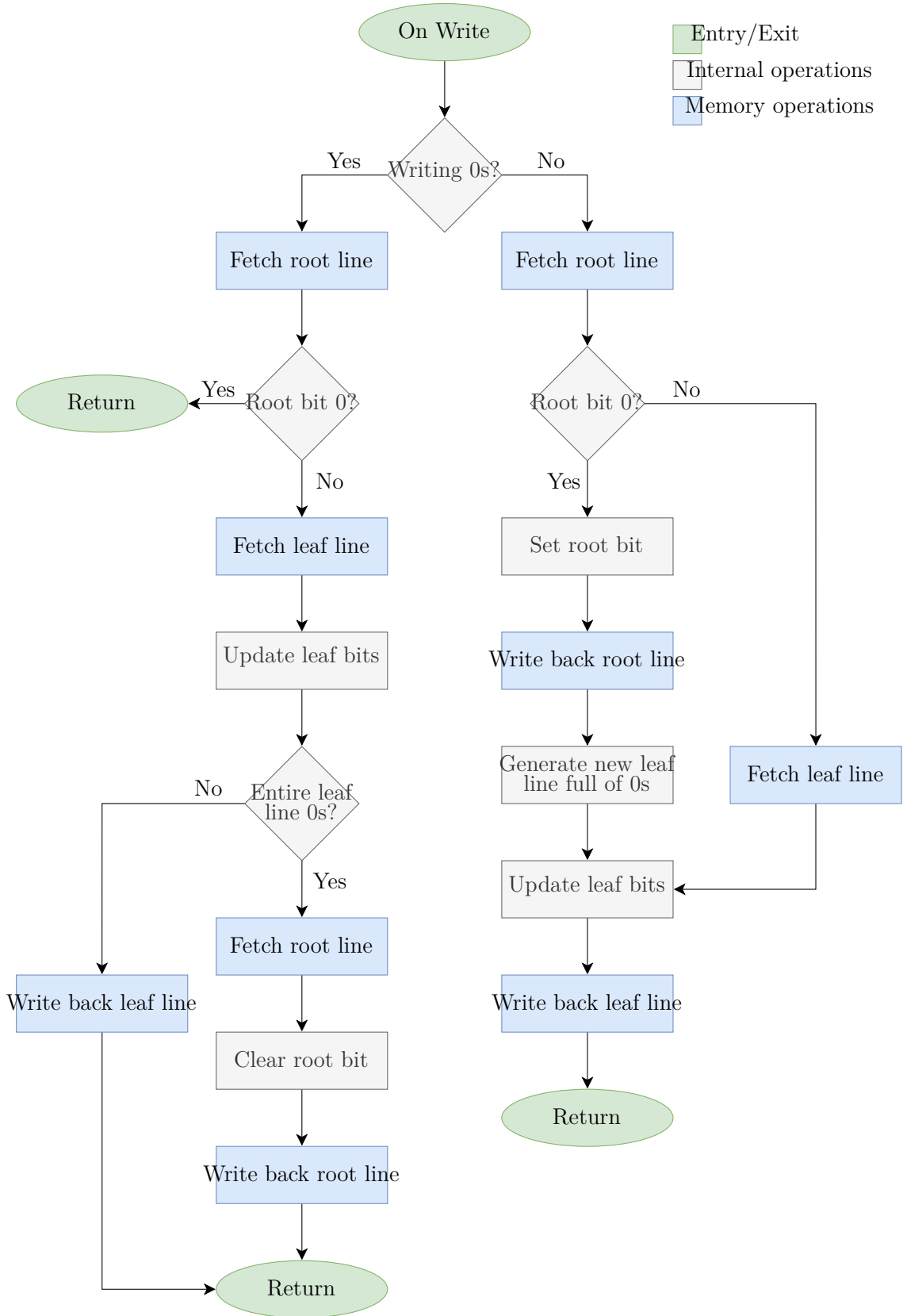


Figure 2.10: The ETM algorithm for writing tags.

The tag cache may hold lines from either the root table or the leaf table. For each *cleared bit from the root table* that resides in the tag cache, we are essentially containing

information about the state of an entire leaf cacheline whilst only consuming one bit of cache space. When executing typical CHERI workloads, it is common to find cleared root bits. It is even common to find entire cachelines of cleared root bits. This is due to the nature of these workloads – the capabilities tend to be focused into small regions of memory, rather than spread out across the entire address space.

The tag cache is even more important here, as these algorithms contain more pairs of operations that have data or control dependencies, meaning that there are proportionally less opportunities to pipeline DRAM accesses, despite potentially having to make more of them in total.

Thankfully, though, this scheme observes a very high hit rate for the tag cache. Thus, whilst in some cases we triple the number of memory accesses for a given LLC miss, the paper finds that there is only a very small overall increase in DRAM traffic overhead, due to the tag cache being able to hold plenty of information about the tags, and it therefore being scarcely necessary to perform additional DRAM accesses. Part of my project is to reproduce these results.

In DRAM, the root table lives below the leaf table. The leaf table lives below the value portion of DRAM (see figure 2.11).



Figure 2.11: The physical layout of the hierarchical table.

Suppose again that the last-level cache misses the cacheline with logical base address a . The controller uses the following formulae to calculate which actual physical addresses it might need to access in DRAM:

$$\text{Actual Value Cacheline Address} = a + \frac{\text{Physical Memory Size}}{65536} + \frac{\text{Physical Memory Size}}{128} \quad (2.4)$$

$$\text{Actual Leaf Tag Cacheline Address} = \left\lfloor \frac{a}{128} \right\rfloor_{64} + \frac{\text{Physical Memory Size}}{65536} \quad (2.5)$$

$$\text{Actual Root Tag Cacheline Address} = \left\lfloor \frac{a}{65536} \right\rfloor_{64} \quad (2.6)$$

The formulae for the actual base addresses of the value cacheline and the leaf tag cacheline are as they were in the simple scheme, with an additional constant of $\frac{\text{Physical Memory Size}}{65536}$ added on. This is to adjust for the offset that the root table adds.

The actual base address of the root tag cacheline will index into the root table. The address is calculated by similar logic to that of the leaf tag, and equation 2.7 explains the presence of the constant 65536.

$$\frac{512 \text{ leaf tags per leaf cacheline}}{4 \text{ tags per value cacheline}} \times 1 \text{ leaf cacheline per root tag} \times 512 \text{ root tags per root cacheline} \\ = 65536 \text{ value cachelines covered by one root cacheline} \quad (2.7)$$

2.1.6 The Morello Project Scheme

As I mentioned prior, during my internship with Arm over the previous summer I contributed to their Morello project. Morello is a research project that defines a microarchitecture based on the CHERI model. Another tag controller scheme that my project focuses on is the scheme used in the Morello microarchitecture. This is due to my own and my supervisor’s familiarity with CHERI, and the Morello architecture and microarchitecture. There is more information about the Morello project at the project’s website [7].

This scheme aims to do a similar thing to the one described in *Efficient Tagged Memory*, but in a different way. Instead of introducing a new, more information-dense table that can be stored in the existing tag cache, it introduces a new tag cache that holds information-dense data.

This scheme uses a simple flat DRAM tag table, just like the one in the basic scheme, but it employs two tag caches. One is a cache reserved for holding tag cachelines that contain *at least one* set tag, call this the non-zero cache. The other is a cache reserved for holding tag cachelines that contain *no* set tags – i.e. lines that are fully cleared, call this the zero cache. The former cache is just like a regular cache; for each cacheline, there is an entry in the cache that stores both the data at the address and the *address-tag** of the cacheline’s base address. However, the latter cache only needs to store the address-tag of the cacheline’s base address, as it must be that the entire cacheline contains zeroes, given that it is in the zero cache, thus there is no need to store any actual data. Requests can be issued to and handled by the two caches in parallel, further mitigating overhead.

* The *tag of a cacheline’s base address* is defined as the upper K bits of the base address of the cacheline (for some K defined by various system parameters). This is the academic-standard name for this value, but it unfortunately clashes with the tagged memory *tags*. To disambiguate, I use the term *address-tag* hereon to refer to the former, and simply *tag* for the latter.

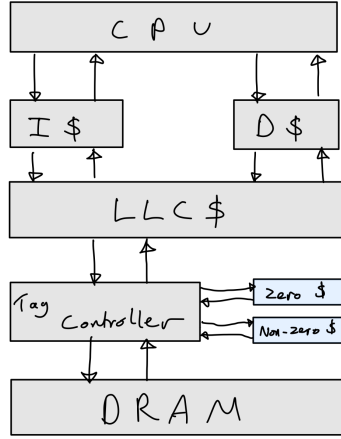


Figure 2.12: The layout of the two tag caches in the Morello memory system.

As the tag table has not changed from the simple scheme, the formulae used by the controller for translating addresses are the same (equations 2.1 and 2.2), and so are the two algorithms (figures 2.5 and 2.6).

2.2 Three-Stage Simulation

Part of my project was to develop a simulator that sits in the middle of a three-stage simulation pipeline. The full pipeline enables the simulation and analysis of the tag table and cache behaviour of given CHERI tag controllers.

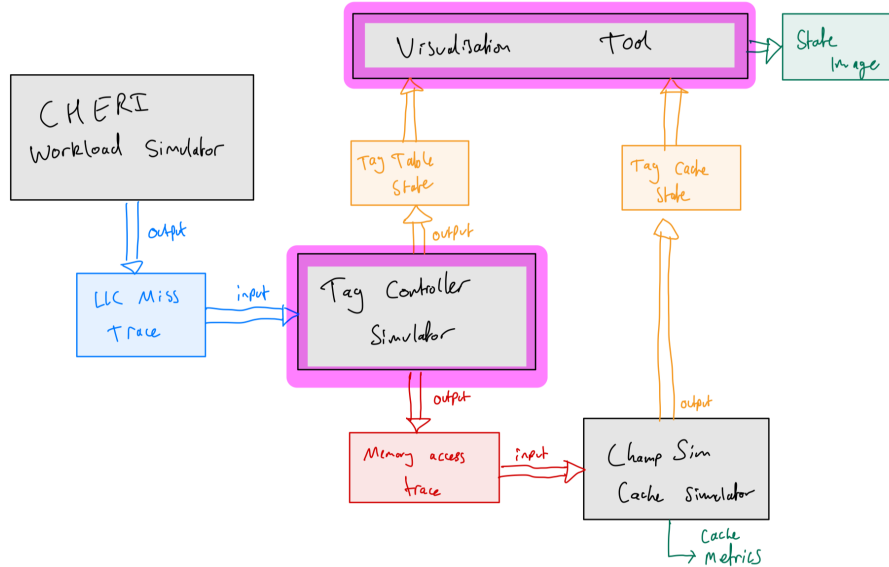


Figure 2.13: A black-box diagram explaining the three stage simulation.

In this section, I explain each of the three stages of simulation.

2.2.1 Workload Execution – LLC Miss Simulation

A former student had previously created a simulator that consumes a program binary and simulates the execution of the binary on a CHERI architecture. The simulator outputs trace files that log the last-level cache misses that occurred during the simulated execution.

I have been given access to a collection of trace files by my supervisor. These files were logged whilst running various benchmark binaries. The simulator I developed takes these trace files as input. As the tag controller only receives information about LLC misses, the trace files contain sufficient information to determine which accesses would be made by the simulated tag controller algorithm if it were to run the given workload.

2.2.2 A Tag Controller Simulator

The main objective of my project was to develop a tag controller simulator in C++. When given a workload it should simulate the memory accesses that a specified tag controller algorithm would result in. The idea is that this simulator can be leveraged to provide insight into the performance and inner workings of the tag controller algorithm being simulated, providing useful information to an architect.

2.2.3 ChampSim – An Appropriate Cache Simulator

ChampSim is an open-source trace based cache simulator written in C++. There is more information about ChampSim at the ChampSim GitHub profile page [8].

My simulator interfaces with ChampSim, having it handle the simulation of the tag cache(s). My simulator provides ChampSim with a trace file of memory accesses that the simulated tag controller makes during its execution of the workload. ChampSim then provides the user with metrics regarding cache access frequency and hit ratio. The user can specify parameters of the tag cache such as cache size and number of ways.

ChampSim is a good choice of cache simulator because its open-source C++ code allowed me to easily extend it to add necessary instrumentation. Also, it has built-in cache prefetchers which can be enabled or disabled simply by adjusting a configuration file. This lets an architect see how much difference adding a prefetcher to their tag cache would make to performance.

I opted to use an existing cache simulator instead of developing my own because the focus of this project is on investigating tag controllers, as opposed to caches in general, and my supervisor informed me that there are likely many opportunities for bugs to arise when developing a cache simulator of my own, and thus it is a more appropriate use of my time to focus more on tag controller research rather than debugging software that has already been written.

2.2.4 Visualisation Tool

It would be useful for an architect to be able to somehow visualise how the state of the tag table and the tag cache changes as the workload executes. Having this insight would

inform new approaches and justify design choices for a novel tag cache.

As part of my project, I developed a tool that allows a user to take ‘snapshots’ of the tag cache at chosen points of workload execution, and view them afterward. The images depict the state of the tag table and the tag cache at that point of execution.

2.2.5 Requirements Analysis

There are several requirements that a successful simulator must meet:

- It must be able to decode the LLC miss input trace file format.
- It must be able to correctly simulate the sequence of memory accesses that the simulated tag controller algorithm would perform when executing the given workload.
- It must be able to produce an accurate output trace file of these accesses that is decodable by ChampSim.

On top of this, for the visualisation tool to be functional and useful, the following are required:

- My simulator must be able to output the state of all tags at given points of execution.
- The system must be able to extract the state of the tag cache from ChampSim at given points of execution.
- The visualisation tool must be able to read in the captured states of the tag table and the tag cache and produce an image that accurately depicts this state.

2.3 Software Engineering Practices

Throughout implementation, I employed good software engineering practices to minimise unnecessary work and maximise the quality of both my final product and my evaluation.

2.3.1 Software Engineering Methodology

When implementing my programs, I applied the Waterfall model of software development. This model was mainly chosen because testing the individual components of the program in isolation was difficult or infeasible, whilst testing the program as a whole was straightforward, due to the pipeline-like nature of the tag controller simulator. Figure ?? shows the order in which the components were developed, and how the resulting system was tested.

2.3.2 Build system

My project uses the CMake build system. This is a well-supported and well-documented build system that allowed for smooth compilation and also easy extension with libraries, such as GoogleTest, which I used for the microbenchmarks.

2.3.3 File Backups and Version Control

I used a private GitHub remote repository to provide version control and backups of all programs, files and documents related to my project. As I incrementally made changes, I made commits to the branch on which I was working, and I would frequently push these changes to the remote server to minimise the loss if things went wrong on my local machine. Fortunately, I encountered no issues with file loss throughout the project.

2.3.4 Software Licences

ChampSim is distributed under the Apache-2.0 licence. The licence allows free redistribution and modification of the software. My project requires modifications to be made to ChampSim, so, as per the licence, I state that such modifications are made in the README of the repository.

GoogleTest is distributed under the BSD 3-Clause licence.

2.4 Starting Point

The following bullet points detail relevant work completed and preparations made before beginning the project:

- I had read the paper *Efficient Tagged Memory*.
- As I have mentioned, over the summer of 2023 I interned at Arm in the Morello Kernel Team, a team working to port the Linux kernel to Arm’s Morello architecture, which is based on the CHERI model. This gave me a chance to get more familiar with the CHERI model.
- My project uses ChampSim, the open-source cache simulator. I also extend this program by adding logging support, by branching from commit `#2b8d3fc28abb6072d7675228418fa7cfe862d4dc`. I took no part in contributing to any versions of ChampSim prior to this commit.

Chapter 3

Implementation

This chapter describes the work I did for the project. It outlines the structure of the programs I developed, and goes in-depth into how exactly I implemented the described functionality. It also details the process by which I designed my novel tag controller scheme.

3.1 Simulator

The simulator consists of 3 main parts: the trace decoder, the tag controller, and the simulated memory. Figure 3.1 shows how these parts fit into the project's pipeline of information flow.

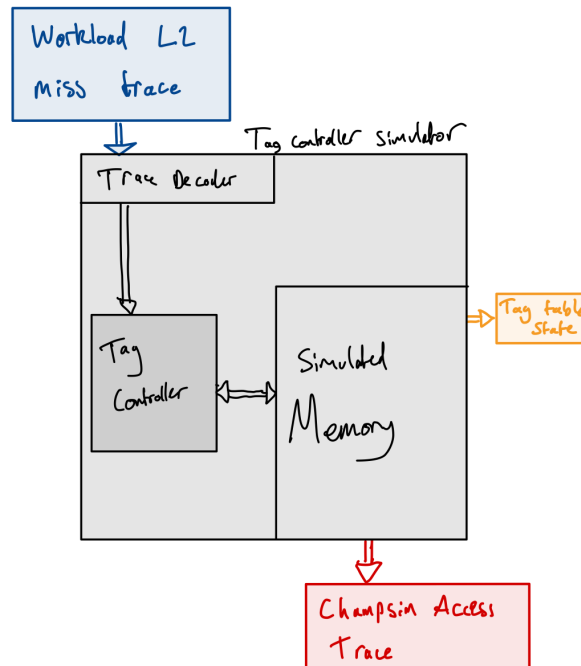


Figure 3.1: An explanation of how the three main parts of the tag controller simulator interact with the inputs and outputs, and with each other.

3.1.1 Simulator Framework

The simulator is a program with a command line interface defined in `main.cpp`. It allows a user to specify the following:

- the name of the tag controller scheme to simulate
- the path of the LLC miss input trace file
- the path of the tag table initial state input file
- the path of the output trace file
- the maximum number of LLC misses to process (optional, default 5 million)
- the path of the output log file to log the tag table state into (optional)
- the indices of instructions after which a tag table state log entry should be made (known as ‘log points’; optional)

`main.cpp` then parses these arguments, and opens the files specified. It creates an instance of a **Decoder** and has it decode the input files. It then creates an instance of a **Controller** of the specified scheme, and a **Simulator**. It then has the **Simulator** simulate the tag controller’s execution when running under the workload of the input trace files.

The **Simulator**’s job is simply to iterate through the decoded LLC misses, passing each to the controller’s `handleRead` or `handleWrite` method, for reads and writes respectively. This emulates the LLC streaming its sequence of misses to the tag controller in a hardware implementation.

Code Snippet 1 Simulator::processTrace

```
1 static size_t processTrace(Decoder &decoder, Controller &controller,  
↪ size_t n, unordered_set<size_t> &log_points) {  
2     size_t processed = 0;  
3  
4     vector<memAccess> buffer(CHUNK_SIZE);  
5     for (size_t at = 0; at < n; at += CHUNK_SIZE) {  
6         size_t read = decoder.readLLCMisses(buffer, min(CHUNK_SIZE,  
↪ n-at));  
7  
8         int i = 0;  
9         for (size_t remaining = read; remaining > 0; --remaining) {  
10             if (buffer[i].type == ACCESS_TYPE_READ)  
11                 controller.handleRead(buffer[i]);  
12             else controller.handleWrite(buffer[i]);  
13  
14             if (log_points.find(processed) != log_points.end()) {  
15                 cout << "LOG @ " << processed << endl;  
16                 controller.dump_table();  
17             }  
18             ++i;  
19             ++processed;  
20         }  
21     }  
22  
23     return processed;  
24 }
```

At the end, `main.cpp` reports the total number of LLC misses that were handled, and the total number of memory accesses the tag controller made. It also saves and closes the output file(s).

3.1.2 Trace Decoder

The `Decoder`'s job is to retrieve information from the input trace files and provide it to whichever calling function requested it.

On its construction, it is given the file objects for the two input trace files. It has two public methods, `readLLCMisses` and `getInitialTags`.

`readLLCMisses` is what the `Simulator` uses to decode LLC misses from the LLC miss input trace file. The file contains a sequence of structs with no padding between them. The structs have fields which begin at specific byte indexes, as detailed in figure 3.2.

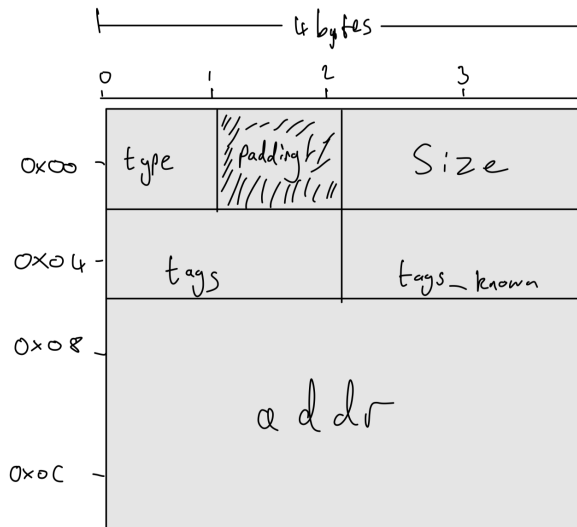


Figure 3.2: The layout of an `llcMiss` struct.

By using the correct format, the decoder reads out the fields of each LLC miss entry from the file, one at a time. For each entry, it constructs an `llcMiss` struct, which is defined in `trace.h`.

The decoder uses `zlib` to read the compressed version of the trace files, rather than requiring the user to first decompress them. This saves them the pains of having to wait for the entire file to be decompressed rather than just the bytes that they are interested in, and of having to store decompressed files on disk whose filesize can be on the order of terabytes.

The final step of this process is to *furnish* the decoded `llcMiss` entry to convert it to a `memAccess` struct (also defined in `trace.h`) before writing that into the buffer. *Furnishing* refers to a process of filling in partially-unknown information about the tags as best we can given the information we do have.

To explain this, we must first understand some background about the source of the input trace files. The CHERI simulator that generated the input trace files worked like so:

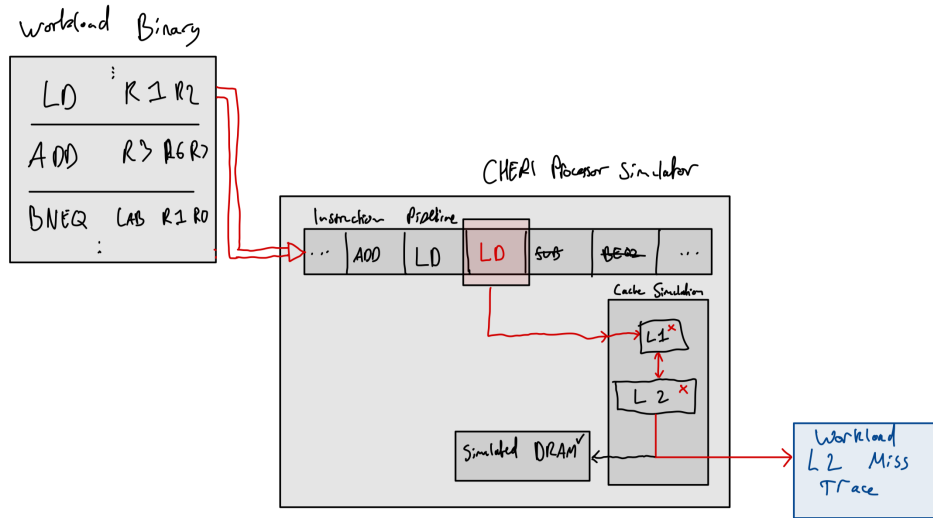


Figure 3.3: An illustration of how the LLC miss files were generated by the CHERI processor simulator.

Instructions were decoded from the program binary, and issued to the simulated CPU. When processing memory access instructions that resulted in LLC misses, an entry was made to the LLC miss trace file, detailing the miss. The entry attempts to describe the state of the four tag bits that were read (in the case of a read), or the state that we update the four tag bits to have (in the case of a write). Figure 3.3 describes this process.

However, the information about the state of *all* four tag bits is not necessarily known at the time the entry was logged. At this time, the simulator only has information about the tag that was read/written by the instruction that caused the miss, and information about the other tags that could be inferred from previous memory access instructions. Figure 3.4 details an example of this. Initially, we know nothing about the state of the tags. Suppose that an instruction sets tag 2 of a cacheline to be high. Suppose further that a later instruction reads tag 3 of this same cacheline; we can know the value of tag 3 from the instruction's result, and we also know that tag 2 is still high, assuming no other instruction has caused it to change since the previous instruction set it. In the case of the other two tags in the cacheline, the simulator does not know what value they have at this time.

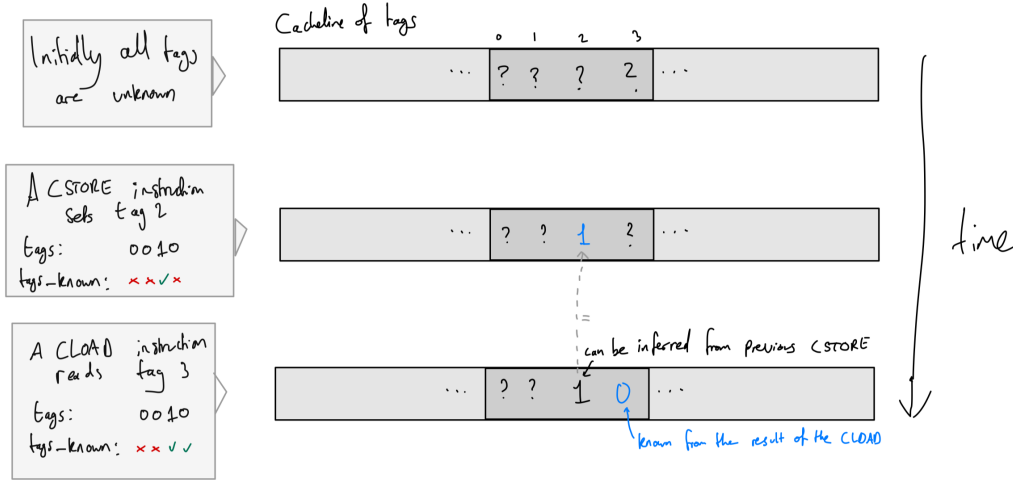


Figure 3.4: An example of how only partial information is known about the state of tags in the table.

This lack of knowledge about particular tags is embedded in each entry in the LLC miss trace file, and upon being decoded, it is stored into the `tags_known` field of the resulting `llcMiss` struct. This field is a bitmask indicating for which of the four tags we have definite information, and for which we do not.

To attempt to gain information for tags that we do not know about, we can refer to the initial state file that is associated with the LLC miss trace. This file contains information regarding what can be known about the *initial state of the tags* in memory. In this file, there is an entry for each tag in the DRAM tag table. The `Decoder`'s private method `getInitialTag` is responsible for making an educated guess at the value of a tag of interest based on the initial state file's entry for that tag.

The `getInitialTag` method first reads and decodes the entry for the tag of interest, obtaining two fields: a boolean named `tag` and a value of the enumerated type `initialAccessType`, which is defined in `trace.h`. The following table details how the `assumeTag` function generates a best-guess initial tag value from these two fields:

Initial access type	Tag	Explanation	Assumed tag
INITIAL_ACCESS_TYPE_CLOAD	0	At some point, before the tag was modified, a capability-load instruction read this tag's value as a 0, so the tag was definitely initially 0.	0
INITIAL_ACCESS_TYPE_CLOAD	1	At some point, before the tag was modified, a capability-load instruction read this tag's value as a 1, so the tag was definitely initially 1.	1
INITIAL_ACCESS_TYPE_CSTORE	0	A capability-store instruction overwrote this tag's value to 0 before the initial value was discovered. We assume the capability-store did not change the value.	0
INITIAL_ACCESS_TYPE_CSTORE	1	A capability-store instruction overwrote this tag's value to 1 before the initial value was discovered. We assume the capability-store did not change the value.	1
INITIAL_ACCESS_TYPE_LOAD	0/1	No capability instructions accessed this tag so we have no concrete information, but it is likely that the value associated with this tag is non-capability data, and thus has its tag cleared.	0
INITIAL_ACCESS_TYPE_STORE	0/1	No capability instructions accessed this tag so we have no concrete information, but it is likely that the value associated with this tag is non-capability data, and thus has its tag cleared.	0

Figure 3.5: The rubric for assuming the value of an unknown tag.

Putting it all together, the `furnish` method converts decoded `llcMiss` structs into `memAccess` structs by using the `getInitialTag` method to fill in gaps. The `getInitialTag` method in turn uses the above assumption scheme defined by `assumeTag` to make an educated guess at what the tag previously was, though at times it must make assumptions due to lack of information.

The `Decoder` class also has a `getInitialTags` method that produces an entire cacheline's worth of initial values of tags in the tag table. It uses the same methods, and the same assumptions for making educated guesses, as the `getInitialTag` method does. It is used by `Controllers` when they initialise the state of their `Memory`.

This method is more efficient than simply calling `getInitialTag` 512 times as it performs a single disk read to retrieve all 512 relevant entries from the initial state file at once. It

also provides a more useful interface to the calling methods, as they operate on cacheline-granularity rather than bit-granularity.

3.1.3 Simulated Memory

`Memory.h` defines the `Memory` class, which represents the tag table stored in memory. This class is responsible for the following:

- Providing an interface to the tag controller, allowing it to read from and write to the state of the tag table
- Keeping track of the state of the tag table as the simulation progresses
- Logging all memory accesses that the tag controller makes

The class does **not** keep track of value data – it only records tag data. This simulation is not concerned with values; only tags.

The `Memory` keeps track of the state of each cacheline in the tag table by using a `std::map` that maps simulated physical addresses to `Cachelines`. A `std::map` is an appropriate data structure for two reasons: 1) it has $O(1)$ lookup time, which is key for providing fast simulation runtimes due to the frequency of tag table accesses and the large size of the table, and 2) it maintains a total ordering of its entries over their keys, meaning that when we dump the state of the tag table to the output log, we can guarantee that we traverse the bytes in order of address. An array would also provide these properties, but in cases where we might want to divide the table into chunks that are spread across the entire physical address space, an array would either need to reserve space that spans this entire range, or have a programmer-defined translation scheme for translating addresses into array indices. Reserving space for the entire physical address space is wasteful as only a very small fraction of it would be used, and a programmer-defined translation scheme adds unnecessary complications for the programmer. On the other hand, a map only needs to have as many entries as the number of cachelines that the tag table can store, and no extra translation scheme needs to be defined.

A `Cacheline` is a C++ alias for the type `std::array<uint8_t, 64>`, representing the 64 bytes that a real cacheline has.

The `Memory` provides a `doRead` method and a `doWrite` method to the tag controller, allowing it to read and write cachelines to the simulated tag table. It is important to remember that this simulation is not concerned with whether the tag cache would hit or miss when accessing memory. The purpose of this simulation is to determine the ***sequence of memory addresses that the tag controller attempts to access*** when running under the chosen workload, which is independent of whether or not the cache hits or misses, thus the control flow of the tag controller’s algorithm will always be accurate to a real run, and therefore the output trace produced will still be an accurate trace of the sequence of memory address accessed by a real controller.

After this simulator produces the access trace, we can feed it into `ChampSim`. `ChampSim` then accurately simulates the physical cache, simulating which lines would hit, which

would miss, which would be evicted, and so on, all the while recording statistics such as access count and hit count.

Each time the tag controller needs to read a tag cacheline from memory, we want to create an entry into the output trace reflecting this. As well as allowing the tag controller to interact with the state of the tag table, the `doRead` and `doWrite` methods also take the responsibility of writing the addresses accessed to the output trace. They do this by calling the private `traceRead` and `traceWrite` methods, which simply append a populated `champsimInstr` struct to the file, building up a trace file that ChampSim can parse.

There are also cases where we want to read from or update the state of the tag table without writing an entry to the output trace file, which I mention later. The `peek` and `set` methods allow this ‘silent’ interaction with the tag table.

The `Memory` class has two getter methods – `getNumAccesses` and `getPrevLogged`. The former returns the number of entries made into the output trace file. The latter returns a vector of all entries that have been made – this is used when testing.

The class also has a method `dump_table`, which writes each bit of the current state of the tag table into the tag table state output log file.

3.1.4 Tag Controller

Code Snippet 2 Controller class

```
1  class Controller {
2
3  protected:
4      Memory memory;
5
6  public:
7      explicit Controller(gzFile output_trace, ofstream &output_log) :
↪      memory(output_trace, output_log) {}
8
9      virtual void setup(Decoder &decoder) = 0;
10
11     virtual uint16_t handleRead(memAccess ax) = 0;
12
13     virtual void handleWrite(memAccess ax) = 0;
14
15     void dump_table() {memory.dump_table();}
16
17     int getNumAccesses() {return memory.getNumAccesses();}
18
19     vector<champsimInstr> getPrevLogged() {return
↪     memory.getPrevLogged();}
20
21     virtual void reportStats() {}
22
23     virtual ~Controller() = default;
24 };
```

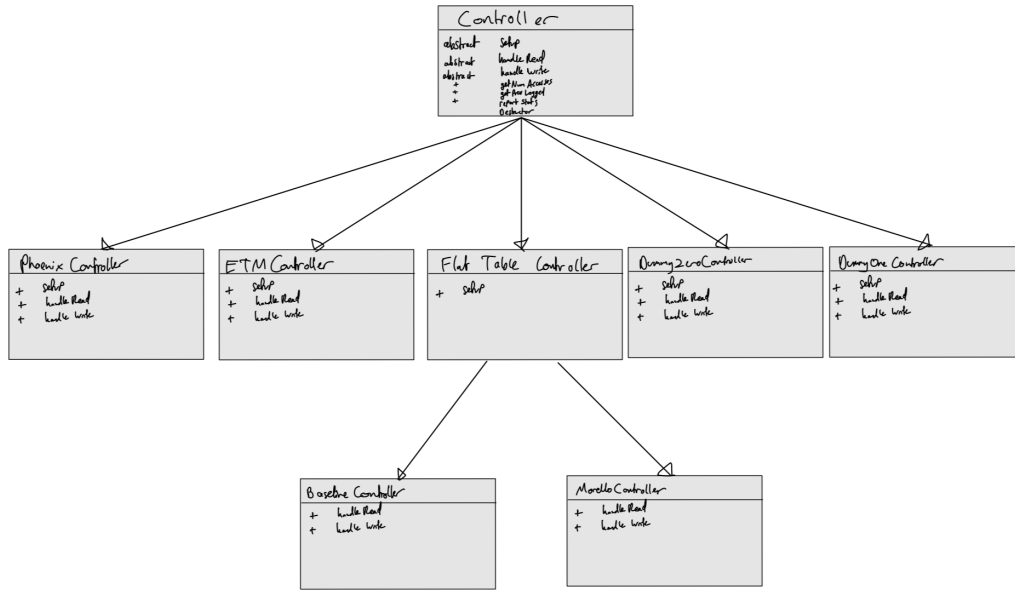


Figure 3.6: The UML diagram detailing the inheritance of the tag controller classes.

A tag controller is implemented as a concrete class that derives from the abstract **Controller** class. Each tag controller has a **Memory**, to which it sends requests during its operation.

Being abstract, this class also details the methods that a concrete tag controller must implement: **setup**, **handleRead** and **handleWrite**.

The **setup** method is to be called immediately after the **Controller** instance is constructed, so that the state of its **Memory** is correctly initialised. This is done with help from the passed **Decoder**, which reads from the initial tag table state input file. This method is abstract because different tag controllers are part of different schemes, and different schemes have different tag table layouts (the Morello Project’s flat table layout from the *Efficient Tagged Memory*’s hierarchical table layout, for instance), thus the algorithm to initialise them will differ.

The **handleRead** and **handleWrite** methods implement the algorithms that the tag controller follows when handling LLC misses. As this simulator is only concerned with tag data and not value data, a concrete implementation of this method only needs to perform the tag table accesses of the algorithm, and not the value access, or the merging of values with tags.

3.1.4.1 Flat Table Controllers

FlatTableController.h defines an abstract class **FlatTableController** that extends **Controller**. This class partially defines a specific type of tag controller – those that use a flat table, such as the controllers from the basic scheme and the Morello Project scheme. Controllers of this type have two things in common: how logical addresses are translated into actual ones, and how the tag cache should be initialised at the beginning of a simulation.

Code Snippet 3 FlatTableController::translateToTagAddr

```
1  static std::pair<uint64_t, uint16_t> translateToTagAddr(uint64_t
   ↪  addr_base) {
2      uint64_t tag_base = ((addr_base / 128) >> 6) << 6;
3      uint16_t tag_offset = ((addr_base>>6)*4) % 512;    // index into
   ↪  the tag cacheline
4      return {tag_base, tag_offset};
5  }
```

The translation from a logical cacheline base address to the corresponding actual tag cacheline base address is detailed in equation 2.2. FlatTableController’s translateToTagAddr method does exactly this (see listing 3).

For a flat table controller, the tag cache is initialised by simply iterating across the entire initial tag table state and populating the cache’s cachelines.

By implementing a single abstract class from which several other classes can inherit instead of duplicating the functionality across several base classes, we reduce the size of the codebase and increase maintainability due to only having to change a single implementation when changes are necessary. It also makes it easier for an architect to implement their own flat table controller, as they already have the abstract class as a starting point. Additionally, it becomes true that by testing this functionality for a single base class, we test it for all of them simultaneously.

3.1.4.2 Basic Scheme

BaselineController.h defines a concrete class BaselineController that implements the tag controller from the basic scheme. The class inherits from FlatTableController.

The implementations of handleRead and handleWrite are very simple, and it is clear how the code directly corresponds to the algorithms from figures 2.5 and 2.6.

Code Snippet 4 BaselineController class

```
1  class BaselineController : public FlatTableController {
2  public:
3      BaselineController(gzFile output_trace, ofstream &output_log) :
4      ↪ FlatTableController(output_trace, output_log) {}
5
6      uint16_t handleRead(memAccess ax) override {
7          pair<uint64_t, uint16_t> result;
8          uint64_t leaf_base_addr;
9          uint16_t leaf_cacheline_index;
10         Cacheline leaf_line;
11
12         result = translateToTagAddr(ax.addr);
13         leaf_base_addr = result.first;
14         leaf_cacheline_index = result.second;
15         leaf_line = memory.doRead(leaf_base_addr);
16         return getTags(leaf_line, leaf_cacheline_index);
17     }
18
19     void handleWrite(memAccess ax) override {
20         uint64_t leaf_base_addr;
21         uint16_t leaf_cacheline_index;
22         Cacheline leaf_line;
23
24         auto result = translateToTagAddr(ax.addr);
25         leaf_base_addr = result.first;
26         leaf_cacheline_index = result.second;
27
28         leaf_line = memory.doRead(leaf_base_addr);
29         modifyTags(leaf_line, leaf_cacheline_index, ax.tags);
30         memory.doWrite(leaf_base_addr, leaf_line);
31     }
32 };
```

3.1.4.3 *Efficient Tagged Memory*'s Hierarchical Scheme

ETMController.h defines a concrete class ETMController that implements the tag controller from the scheme described in *Efficient Tagged Memory*.

This class's setup method instantiates the *leaf* table in the same way that a flat table controller instantiates its table. Whilst it does this however, it also populates its *root* table, byte by byte. Each time a cacheline of tags is placed into the leaf table, we keep track of whether or not the cacheline contained any set tags, recording a 1 in the current

byte if there were any, and a 0 if not. After processing 8 leaf table cachelines, we push the current byte into the current cacheline. After pushing 64 bytes, we **set** the now-completed cacheline in the root table.

Code Snippet 5 ETMController::setup

```

1  void setup(Decoder &decoder) override {
2      Cacheline root_line, leaf_line;
3      uint8_t root_byte;
4      bool no_leaves_set;
5
6      for (uint64_t rl = 0; rl < ROOT_TABLE_SIZE; rl +=
↪ TAG_CACHE_LINE_SIZE) {
7          for (int rb = 0; rb < TAG_CACHE_LINE_SIZE; ++rb) {
8              root_byte = 0;
9              for (int rt = 0; rt < 8; ++rt) {
10                 root_byte <= 1;
11
12                 // focus on this single root tag
13
14                 decoder.getInitialTags((8*rl+8*rb+rt)*TAG_CACHE_LINE_SIZE*8,
↪ TAG_CACHE_LINE_SIZE*8, leaf_line);
15
16                 memory.set(TAG_CACHE_LINE_SIZE*(8*rl+8*rb+rt) +
↪ ROOT_TABLE_SIZE, leaf_line);
17
18                 no_leaves_set = isClear(leaf_line);
19                 root_byte |= no_leaves_set ? 0 : 1;
20             }
21             root_line[rb] = root_byte;      // push the root byte into
↪ the line
22         }
23         memory.set(rl, root_line);      // insert the root line into
↪ the cache
24     }
25 };

```

This class defines two auxiliary functions, `translateToLeafAddr`, and `translateToRootAddr`. Just as `FlatTableController`'s `translateToTagAddr` implemented the appropriate equation from the Preparation chapter to translate a value cacheline's logical base address to the corresponding tag cacheline's actual base address, `translateToLeafAddr` implements equation 2.5 to translate a value cacheline's logical base address to the corresponding tag cacheline's actual base address in the *leaf table*. Similarly, `translateToRootAddr` implements equation 2.6 to translate that logical base address to the actual base address of the corresponding cacheline in the root table instead

– that is the base address of the cacheline in the root table that contains the root bit whose scope covers the corresponding leaf table line.

Code	Snippet	6	ETMController::translateToLeafAddr	and
ETMController::translateToRootAddr				

```

1  static std::pair<uint64_t, uint16_t> translateToLeafAddr(uint64_t
   ↪  addr_base) {
2      uint64_t leaf_base = (((addr_base / 128) >> 6) << 6) +
   ↪  ROOT_TABLE_SIZE;
3      uint16_t leaf_offset = ((addr_base>>6)*4) % 512;           // index into
   ↪  the tag cacheline
4      return {leaf_base, leaf_offset};
5  }
6
7  static std::pair<uint64_t, uint16_t> translateToRootAddr(uint64_t
   ↪  addr_base) {
8      uint64_t root_base = ((addr_base / 65536) >> 6) << 6;
9      uint16_t root_offset = ((addr_base>>6)*4 / 512) % 512; // index into
   ↪  the tag cacheline
10     return {root_base, root_offset};
11 }

```

The implementations of `handleRead` and `handleWrite` are more complex than that of the basic tag controller, but once again the code directly corresponds to the algorithms from the appropriate figures (2.9 and 2.10).

3.1.4.4 The Morello Project Scheme

`MorelloController.h` defines a concrete class `MorelloController` that implements the tag controller from the Morello Project scheme. The class inherits from `FlatTableController`.

The Morello project tag controller implementation is a special case. As mentioned in the Preparation chapter, the Morello Project’s microarchitecture uses two distinct caches. My program handles this by producing the access logs for the two caches in two separate simulations. On invoking the program, the user specifies whether they want to simulate the zero cache, or the non-zero cache. The simulator will then run and produce an output trace file that logs *only the accesses made to the specified cache*. For example, if the user specifies that they want to simulate the non-zero cache, then the output trace produced will detail all accesses made to *non-zero* tag cachelines, and not any to *zero* cachelines. This way, the simulation framework does not need to be adapted to handle multiple caches at once, and the output log can still be passed straight to `ChampSim` to simulate the chosen cache.

Now though, this means that the algorithms that we implement must change. In simulation, we want the **Memory** to only log a memory access to the output trace if the line that the controller is accessing is a line that *would be held in the cache we are simulating*. To do this, we need a way for the controller’s algorithm to know the state of the cacheline before it decides whether or not to log the memory access. The **Memory**’s **doRead** would tell the controller the state of the cacheline, but as it does this, it has the side-effect of logging the memory access to the output trace file, which is a problem if the line happens to be one that we do not want to log the access of. That is, with this method, we can only know if we actually want to log the access *after* we have called the method and consequently already logged the access.

The solution is to instead use the **Memory**’s **peek** method. With this method the controller can see the state of the cacheline without logging the access, allowing it to decide whether or not to log the access *after* examining the line’s state, and only doing so if it matches the cache being simulated.

Similarly, on updating a tag, we might not necessarily want to log the write memory access, but it is important that we update the state of the tag in the simulation, otherwise the algorithm might later read stale data and make decisions that are not accurate to a real world run of the controller, where the value would have changed in memory. The **Memory**’s **set** method can be used to update the state of a cacheline without logging a write memory access to the output trace.

Figures 3.7 and 3.8 detail the new algorithms we must implement. They contain new ‘pseudo-memory access operations’ that correspond to **peeking** and **setting**. These operations are purely for simulation control flow purposes and do not correspond directly to real operations that a physical tag controller would perform.

The algorithm for reading is as follows:

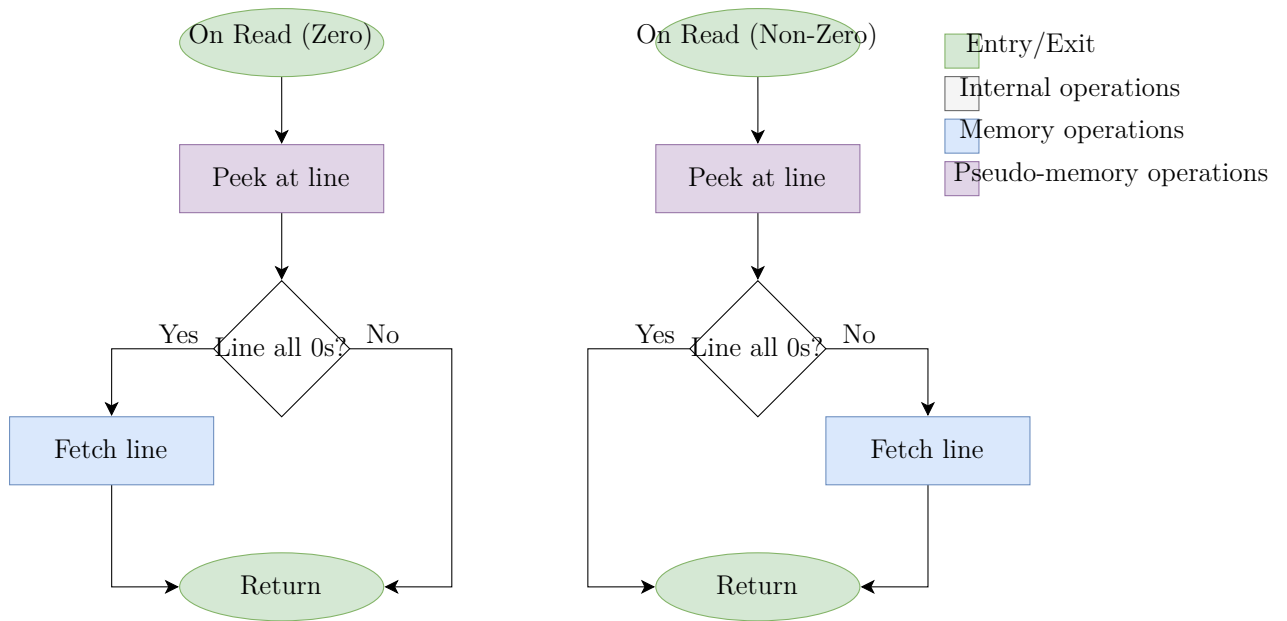


Figure 3.7: The Morello Project algorithm for reading tags. Pseudo-memory access operations are purple.

The algorithm for writing is as follows:

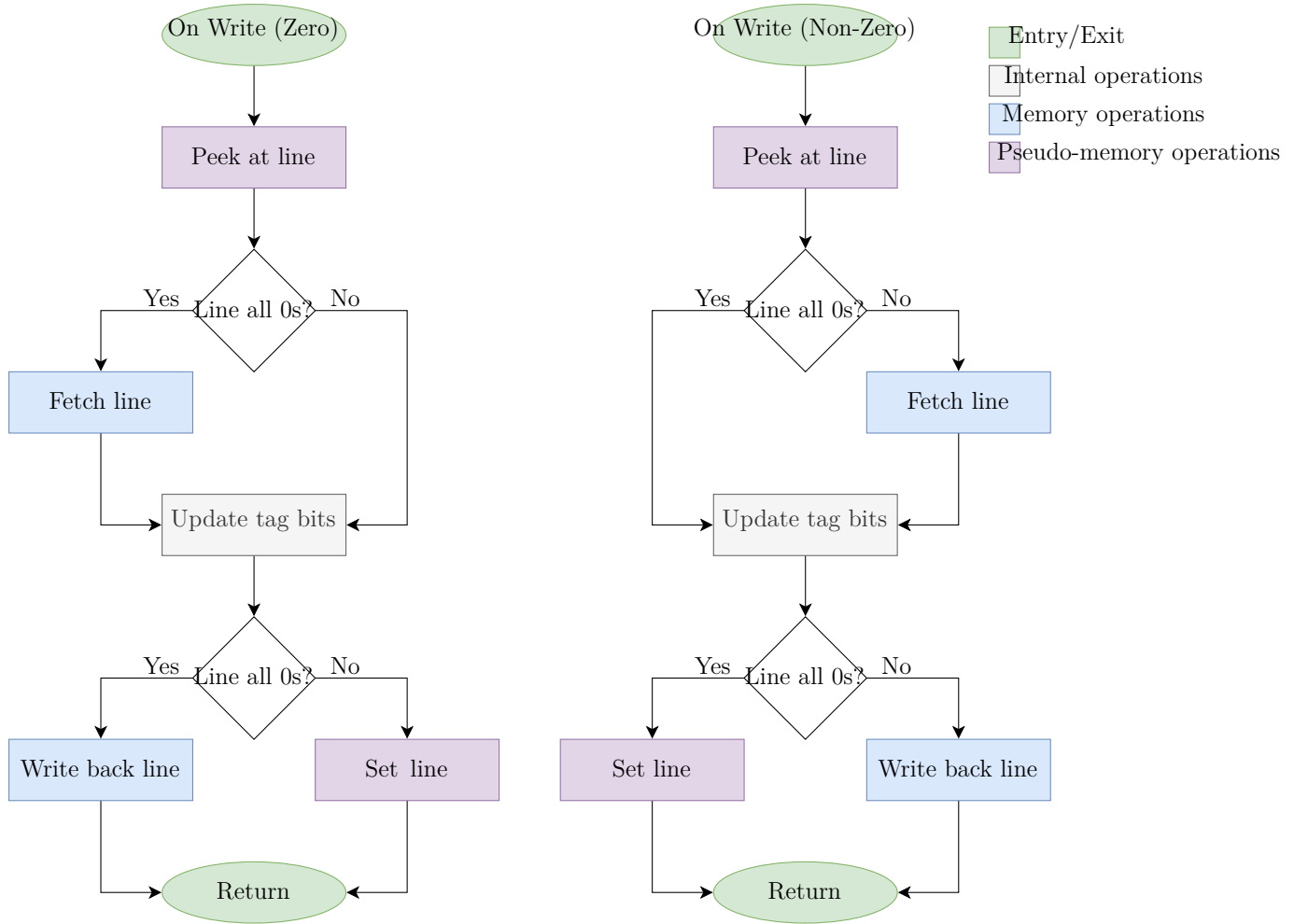


Figure 3.8: The Morello Project algorithm for writing tags.

The implementations of `handleRead` and `handleWrite` implement these algorithms.

3.2 Logging

3.2.1 Logging Tag Table State

Another job of the `Simulator` class is to stimulate the `Memory`, via the `Controller`'s proxy method, to dump its tag table state once a log point is reached.

When `ChampSim` parses our output trace, we want it to log the state of the tag cache at this same point of program execution. However, `ChampSim`'s unit of work is a memory accesses, not an LLC miss. As there may be more than one memory access per the handling of a single LLC miss, we must let `ChampSim` know *after which memory access* (as opposed to after which LLC miss) it should dump its state. Therefore, after the `Simulator` stimulates the memory, it also reports the index of the most recent memory access request, which will later also be given to `ChampSim` for this purpose.

3.2.2 Adding Logging to ChampSim for Tag Cache State

To add cache state logging support to ChampSim, I had to modify three files.

In `cache.h` I simply made the `BLOCK` struct public so it could be accessed from outside of the class.

In `main.cc` I added to option to specify two extra command line arguments:

- the path of the output log file to log the tag cache state into
- the indices of the memory accesses after which a tag cache state log entry should be made

The `main` function passes this information to the `champsim::main` function defined in `champsim.cc`.

This function was then modified to check after each phase of processing if it has reached a log point. If it has, it dumps all of the addresses currently resident in the simulated cache into the output log file.

3.3 Verification

3.3.1 Microbenchmarks – Unit Tests

I implemented some unit tests to test the correctness of the address translations and access sequences of some of the concrete tag controller implementations. They have the controller handle a single access, and then analyse the sequence of addresses that the controller made in response, asserting that the addresses accessed match what was expected. These microbenchmarks were particularly useful to debug the controllers as I implemented them, as their precision highlighted exactly where the issues lay.

3.3.2 Macrobenchmark – Trace Verifier

The input trace files usefully contain information about the results of reads made to the DRAM tag table during the workload simulation. This allows us to cross-reference the results of reads that our implemented tag controllers produce with those found in the trace file. The trace files contain hundreds of millions (if not more) of read instructions, so if of all of the read results match, then we can say with extreme confidence that our implementation of the tag controller is correct. As part of my project I developed a program that allows the user to provide a reference workload and verify the correctness of a tag controller in this way.

Code Snippet 7 verify.cpp's verify function

```
1  int verify(Decoder &decoder, Controller &controller, size_t n) {
2      uint16_t tags, bufferTags;
3      int discrepancies = 0;
4
5      vector<memAccess> buffer(CHUNK_SIZE);
6      for (size_t at = 0; at < n; at += CHUNK_SIZE) {
7          size_t read = decoder.readLLCMisses(buffer, min(CHUNK_SIZE,
↪      n-at));
8
9          size_t i = 0;
10         for (size_t remaining = read; remaining > 0; --remaining) {
11             if (buffer[i].type == ACCESS_TYPE_READ) {
12                 tags = controller.handleRead(buffer[i]);
13                 bufferTags = buffer[i].tags;
14                 if (tags != bufferTags) {
15                     cout << "TAGS DIFFER AT INSTRUCTION INDEX " << i <<
↪      endl;
16
17                     cout << "ADDRESS:      " << buffer[i].addr << endl;
18                     cout << "TRACE TAGS:    " << bufferTags << endl;
19                     cout << "BASELINE TAGS: " << tags << endl;
20                     discrepancies++;
21                 }
22             } else controller.handleWrite(buffer[i]);
23             ++i;
24         }
25         cout << "VERIFY: " << discrepancies << " total discrepancies" <<
↪      endl;
26         return discrepancies;
27     }
```

3.3.3 Macrobenchmark – Cross-Verifier

The program also allows the user to specify a pair of tag controllers, and compare them against each other. It does this by taking an input workload, simulating both of them handling the desired number of LLC misses, and calling `handleRead` on every address in physical memory, ensuring that all tag state is functionally equivalent. As the controllers handle the LLC misses, their `Memorys`' tag table states will update according to the implementations of their algorithms. If at the end the states of all tags in the table match, the user can say with extreme confidence that the inputs, outputs, and logical state transitions of the two tag controller implementations are the same.

With this program, I also implemented two dummy controllers, `DummyZeroController` and `DummyOneController`, which always return zeroes or ones from their `handleRead` methods respectively. By comparing a scheme to either of these, a user can confirm whether or not their controller is only reading out zeroes or ones, which can be useful for debugging purposes.

Code Snippet 8 `verify.cpp`'s `crossVerify` function

```

1  int crossVerify(Controller &controller1, Controller &controller2) {
2      memAccess ax;
3      uint16_t tags1, tags2;
4      int discrepancies = 0;
5
6      for (uint64_t addr = 0; addr < MEMORY_SIZE; addr += 16) {
7          ax = memAccess(ACCESS_TYPE_READ, 64, 0, addr);
8
9          tags1 = controller1.handleRead(ax);
10         tags2 = controller2.handleRead(ax);
11
12         if (tags1 != tags2) {
13             cout << "TAGS DIFFER AT ADDRESS " << addr << endl;
14             cout << "CONTROLLER 1 TAGS: " << tags1 << endl;
15             cout << "CONTROLLER 2 TAGS: " << tags2 << endl;
16             discrepancies++;
17         }
18     }
19     cout << "CROSS-VERIFY: " << discrepancies << " total discrepancies"
↪    << endl;
20     return discrepancies;
21 }

```

3.4 Visualisation Tool

With the ability to capture the state of the tag table and tag cache at any point in the execution of a workload, we can now create a tool to visualise the this state, effectively showing us a ‘snapshot’ of the tag state at that moment in execution.

`visualiser.py` combines the simulation and the visualisation into one program. It allows the user to specify the following command line arguments:

- the name of the tag controller scheme to simulate
- the path of the LLC miss input trace file
- the path of the tag table initial state input file

- the maximum number of LLC misses to process
- the indices of the instructions after which a snapshot should be made
- the string prefix of the output image filenames

The following describes the program's sequence of events:

1. The tag controller simulator is invoked, being passed the arguments it requires.
2. The simulation finishes, and the output files are produced.
3. The standard output of the tag controller simulation is read, collecting the reported memory accesses after which ChampSim must dump the state of the tag cache.
4. The instrumented version of ChampSim is invoked, being passed the required arguments.
5. ChampSim simulates the operation of the tag cache, and logs the state of the cache at the desired points. It then finishes, reporting its usual metrics to the user.
6. Now having the logged states of both the tag table and the tag cache, the program finally generates a PNG that visualises this state. One PNG is generated for each log point.

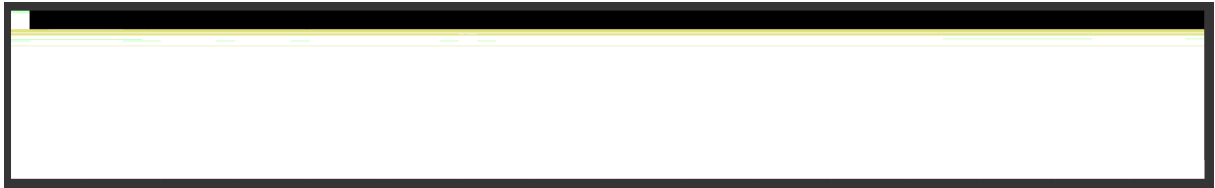


Figure 3.9: A snapshot of the tag table when running the Efficient Tagged Memory scheme.

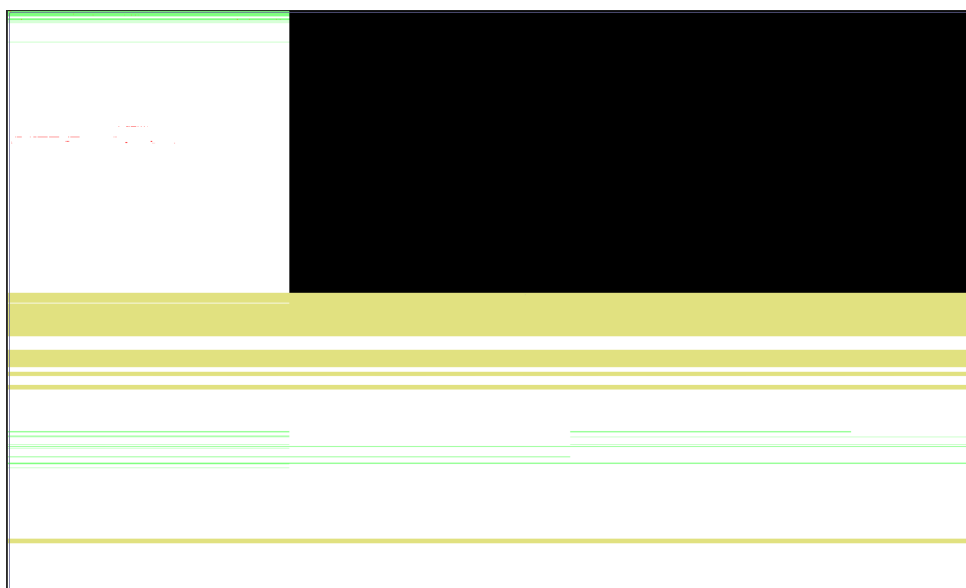


Figure 3.10: A closer look at the root table.

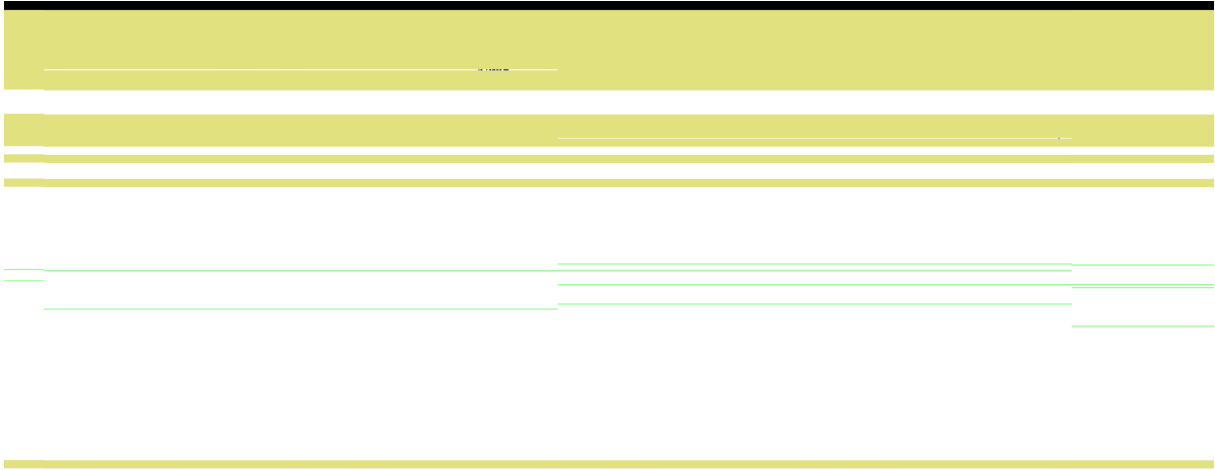


Figure 3.11: A closer look at the leaf table.

3.5 Phoenix – A Novel Scheme

3.5.1 The Motivation

After confirming the correctness of my implementations of the above three algorithms by using the verifiers, I used the visualisation tool to inspect the state of the tag table and cache during a simulation of the *Efficient Tagged Memory* scheme.

I noticed that while some tags were set, there were still huge portions of the table that contained no set tags. In a physical system, these correspond to kilobytes of memory that only contains zeroes. Seeing this suggested two things to me:

The first was that there might be a more efficient way to store all of these zeroes in the tag cache. *Efficient Tagged Memory*'s root table already does a very good job of representing the zeroes succinctly, but there may still be opportunities to increase the cacheability of the tag table in this respect.

The second was that there might be an opportunity to free these empty regions to a DRAM allocator, if the system were running under a hypervisor with such a mechanism. The above three schemes have no means for doing this.

3.5.2 The Intuition

3.5.2.1 Reference Counters

A thought that initially came to my mind to increase the cacheability of the tag table was to increase the scope of the root bits to more than just a single cacheline of leaf tags. Doing this would mean that we could contain a larger portion of the tag table in the tag cache, as each zero root bit would represent a larger section of the cache.

Increasing the scope significantly might also be a start to dynamic DRAM allocation. Current schemes have no notion of how much physical space they currently require for their tag tables, but A scheme that divides its tag table up into large blocks could allocate

and free those blocks on-demand. If an entire block is unused, the space it is consuming could be freed back to the hypervisor's memory pool, to be used by another guest OS. Once the space is required again, the guest OS could request more physical memory.

Increasing the scope of the root bits brings problems, however. One problem is that on writing zeroes into the leaf table, we must check the entire group of tags covered by the corresponding root bit to see if all tags are now zero, so as we appropriately zero the root bit if so. A larger group means more cachelines must be retrieved during this process, increasing latency, polluting the cache, and increasing the chance of a cache miss.

A potential solution to this problem is to use a counter to keep track of the number of tags that are non-zero. Consider a scheme where each group of leaf tags has its own counter, and all of the counters are stored in a table. On setting tags, we increment the counter for that group, and on clearing them, we decrement it. Once the counter reaches zero, we know that all tags in the group must be zeroes, so we can clear the root bit without having to do additional checks. Lines from the counter table could also be held in the tag cache, consuming relatively small amounts of precious cache space, much like the root cachelines.

It is actually more appropriate to instead count the number of non-zero cachelines, instead of non-zero tags. This reduces how frequently we need to update the counter, helping to keep bus traffic down. Checking if the whole cacheline is non-zero instead of just the tag(s) the we update brings no additional memory accesses, as when serving requests to the LLC, we must access the entire cacheline anyway to fulfil the request.

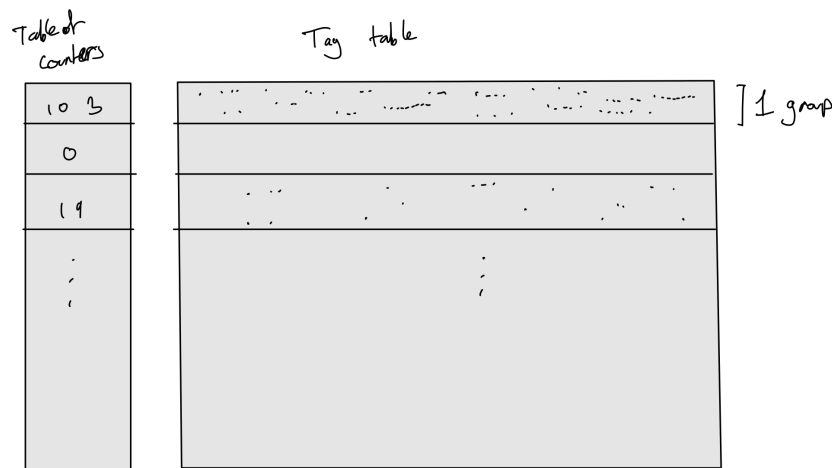


Figure 3.12: An illustration of the table of tag counters.

Suppose that we have M bytes of physical memory in which value data is stored. Let k be the number of tag cachelines in each counter group.

$$\frac{M \text{ value data bytes}}{16 \text{ value data bytes per tag bit}} = \frac{M}{16} \text{ tag bits} \quad (3.1)$$

$$\frac{\frac{M}{16} \text{ tag bits}}{512 \text{ tag bits per tag cacheline}} = \frac{M}{8192} \text{ tag cachelines} \quad (3.2)$$

$$\frac{\frac{M}{8192} \text{ tag cachelines}}{k \text{ tag cachelines per group}} = \frac{M}{8192k} \text{ tag groups, and counters} \quad (3.3)$$

Each counter will require at least $\text{ceil}(\lg(k+1))$ bits to keep track of it, as it must be able to represent all numbers in the range $[0, k]$.

Therefore the minimum extra space required by the table of counters will be $\frac{M}{8192k} \times \frac{\text{ceil}(\lg(k+1))}{8}$ bytes. In practice, padding would be used to keep the counters from crossing word boundaries, consequently increasing the size of the table.

We must now choose an appropriate value for k , the group size. To do this, I used the visualisation tool to investigate the state of the tag table for some workloads. I noticed that the set tags tended to be focused into pages. That is, they tended to have either no tags set, or many tags set; relatively few pages had only a handful of tags set (see figure ??). This suggested to me that setting the group size to equal the size of a page of tags, 4096 bytes (64 cachelines), was a good choice. This way, the majority of the counters will be zero, and the rest will mostly be high numbers. The number is not too small so as to result in there being too many counters, requiring too much extra space and too much additional cache pollution. The number is also not too large so as to merge freeable regions of zeroes with regions containing tags, disallowing us from freeing the entire group. That is, due to the tags being concentrated amongst a small number of pages, we can still free most of the space being used to hold zero tags without having to use smaller granules.

Another advantage of choosing this number is that it is likely that the dynamic DRAM allocator operates on page-granularity, making the interface clean.

So, let $k = 64$. As an exapmle, suppose that $M = 4\text{GiB}$:

We would have $\frac{2^{32}}{8192 \times 64} = 8192$ groups of tags, and also this many counters.

Each counter would require at least $\text{ceil}(\lg(64+1)) = 7$ bits to keep track of it.

Therefore the minimum extra space required by the table of counters will be $8192 \times \frac{7}{8} = 7186 = 7\text{KiB}$.

This is a somewhat significant space overhead, but it comes with the ability to now free pages to DRAM. By freeing only 2 pages, we can reduce the overhead to below that of the other schemes.

Regarding cache polution, only the counters of groups currently in use will need to be kept in the cache, so the effect that the counters have on cache hit rate will be relatively small, but non-negligible.

3.5.2.2 The Superroot Table

I then iterated on this idea. The performance of the scheme detailed in *Efficient Tagged Memory* was thanks to its finely-tuned tag table. Ideally, we would not have to sacrifice

this performance. However, another problem with increasing the scope of the root tags is that it also decreases the chance that all of the leaf tags it covers are zero, and thus we are more likely to need to perform a two-stage lookup, meaning we bring more zeroes into the tag cache than before. That is, the effective cacheability of the table has decreased – whenever we bring in a leaf line of zeroes, we risk evicting a root line, meaning the coverage of the tag cache is reduced. It is a balancing act to find the optimal scope for the root bits, which the *Efficient Tagged Memory* scheme has already calibrated for.

Instead of replacing the root table with a set of counters, we can extend *Efficient Tagged Memory*’s scheme, keeping the root table, but also maintaining page-wide counters to preserve the property of not needing to scan the entire page of leaf tags each time we clear one. If we were to keep the root table and also have counter groups that cover $k = 64$ tag cachelines, there would be $\frac{4096 \times 8}{512} = 64$ root bits for each counter.

By keeping the root table, we can now discard the counters entirely, replacing them with something more physically compact. We know that checking if all the tags in the page are cleared is equivalent to checking if all of the 64 corresponding root bits are clear, and as the 64 root bits fit in a word or two, this process is much more manageable than scanning an entire page, so we can simply do this instead of maintaining page-wide counters.

Instead of counters, we can simply have a single bit that we clear whenever the corresponding 64 root bits are cleared, and we set when any of the 64 root bits are set. This effectively adds a third layer at the top of the hierarchical table. The “superroot” level has one bit for each page of leaf tags. These superroot bits are stored in their own table, beneath the root and leaf tables. By transitivity through the root level, we maintain the property that if any tags in the block are set, the superroot bit will be set, and it will be cleared otherwise.

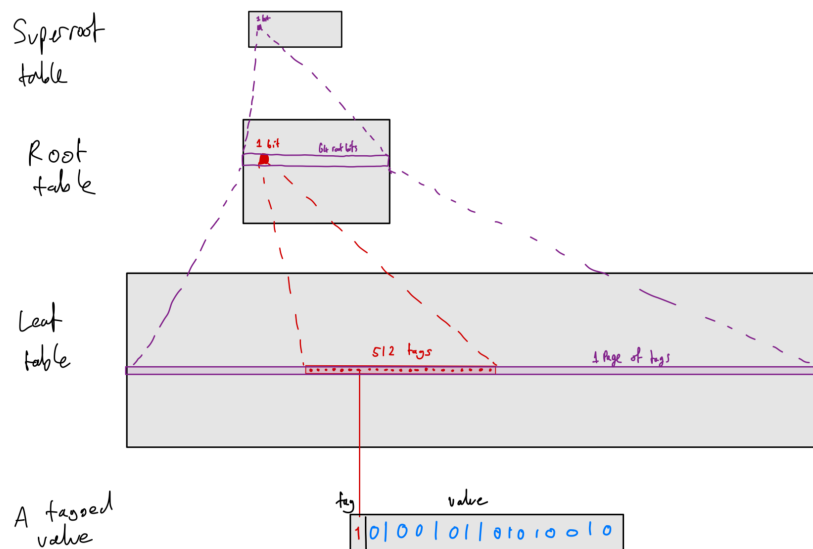


Figure 3.13: The Phoenix scheme’s three-level hierarchical lookup table.



Figure 3.14: The physical layout of the hierarchical table in the Phoenix scheme.

3.5.2.3 The Tag Working Set

In this scheme, we want to free pages of leaf tags that are unused and only contain zeroes. For pages that contain at least one leaf tag set, we must keep them allocated. To do this, we can mandate that whenever a superroot bit becomes zero, we free its corresponding page of leaf tags. On attempting to read the page, we will first consult the superroot bit, and upon noticing that it is zero, we will not attempt to read any deeper into the hierarchy, avoiding encountering a fault on attempting to read an unallocated page.

This would be correct, but it might result in undesirable thrashing of the DRAM allocator for some workloads – if a page only contains one tag that is constantly being set and unset, we would make many requests to the DRAM allocator to free and then re-allocate that page, likely resulting in increased latency experienced within our guest OS, and it has the potential to adversely affect the performance of other guest OSes if they happen to also be queueing to use the allocator.

To mitigate this, we have the concept of a “tag working set”. This is a set of pages in which the system is currently accessing *non-zero* tags. It operates similarly to a cache in the sense that it is a set which respects temporal locality: when we access non-zero tags in a page, we place that page into the tag working set, but the set’s finite size means we must in turn evict another page from the set, *ideally one for which we have not accessed non-zero tags for some time*.

The tag working set can help mitigate the thrashing problem. We can mandate that we only free a page once the following becomes true:

$$\text{The page contains no set tags} \wedge \text{The page is not in the tag working set} \quad (3.4)$$

When revisiting the thrashing example, we notice that we would not free the page immediately on clearing it, as it would still remain in the tag working set for some time. If we tune the parameters well, it will remain in the set until the tag is set once more, and we will then have avoided making any requests the allocator.

With this, we can describe a set of rules for when to update the tag working set, and when to allocate and free pages. Algorithm 9 details these rules.

These rules maintain the following invariant:

$$\forall \text{ Tag Pages } . \text{Page is not allocated} \iff \text{Page is not in the tag working set} \wedge \text{Page has no tags set} \quad (3.5)$$

Algorithm 9 Set of rules for when to update the tag working set, and when to allocate and free pages

```
1: On reading a non-zero tag from a page  $p$  do
2:   if  $p$  is in the tag working set then
3:     Promote  $p$  to the most-recently used in the tag working set;
4:   else
5:     Add  $p$  to the tag working set, marking it as the most-recently used;
6:   end if
7: End
8:
9: On writing a non-zero tag to a page  $p$  do
10:  if  $p$  is in the tag working set then
11:    Promote  $p$  to the most-recently used in the tag working set;
12:  else
13:    Allocate  $p$ ;
14:    Add  $p$  to the tag working set, marking it as the most-recently used;
15:  end if
16: End
17:
18: On clearing a page of tags  $p$  do
19:  if  $p$  is in the tag working set then
20:    Free  $p$ ;
21:  end if
22: End
23:
24: On Evicting a page  $p$  from the tag working set do
25:  if  $p$  contains no set tags then
26:    Free  $p$ ;
27:  end if
28: End
```

Which may be written equivalently as:

$$\forall \text{ Tag Pages } . \text{ Page is allocated } \iff \text{ Page is in the tag working set } \vee \text{ Page has at least one tag set} \quad (3.6)$$

3.5.3 The Algorithm

The two sections above describe the workings of the novel tag controller scheme that I devised. I named it the Phoenix scheme.

We now need an exact algorithm for reading from and writing to the tag table.

The following is the algorithm for reading:

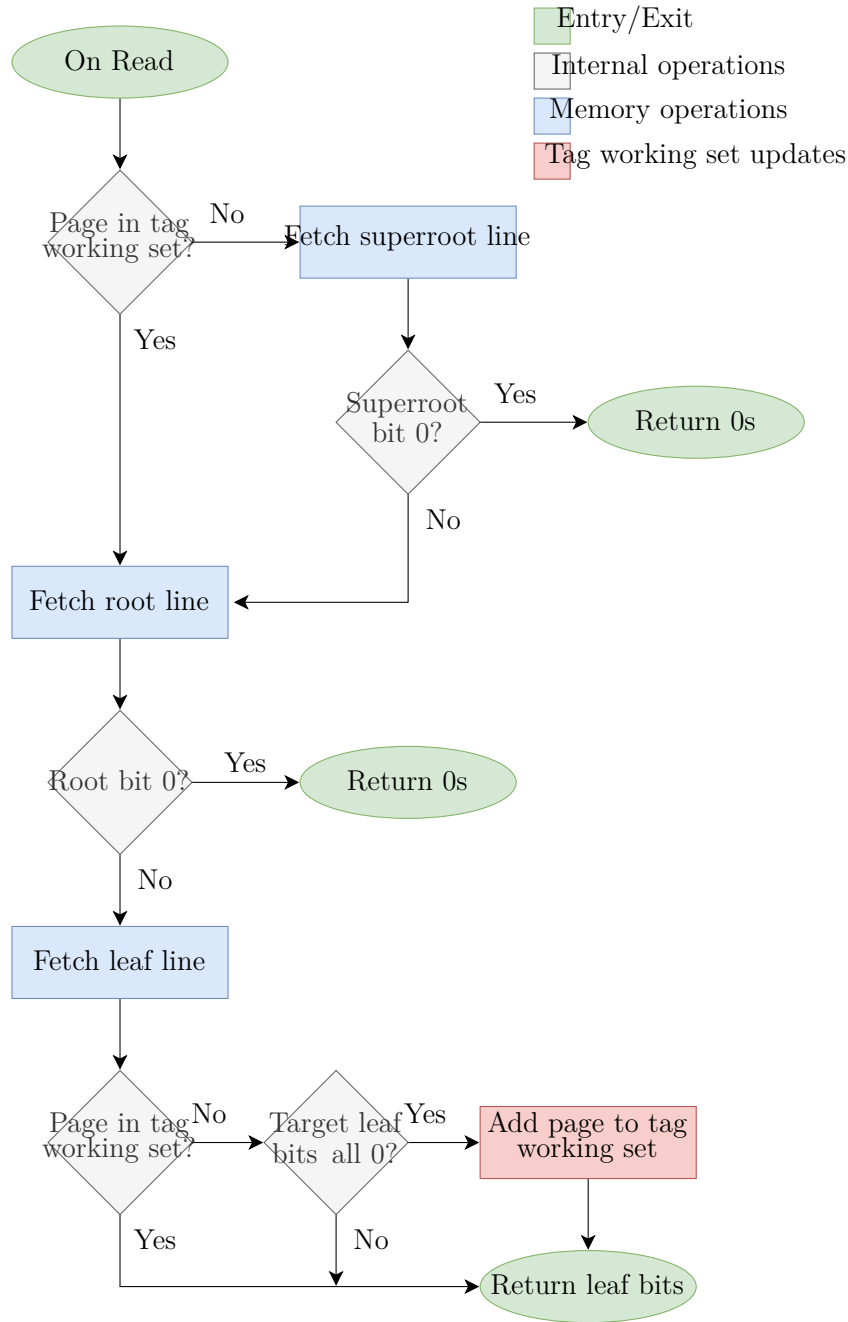


Figure 3.15: The Phoenix algorithm for reading tags. Tag working set operations are in red.

Introducing the tag working set has a secondary benefit. A design choice I made in the reading algorithm was that if the page we are reading is in the tag working set, we skip reading from the superroot table and instead read straight from the root table. Pages that are in the tag working set are most likely to have their superroot bits set, as after all it is *by accessing a set tag in the page* that a page enters the set. This way, we avoid the extra third lookup for the cases where it is most likely that this lookup will not allow us to take a shortcut. As we skip straight to the root table, this means that in these cases the lookup speeds cannot be much worse than those experienced in the *Efficient Tagged Memory* scheme.

It is always safe to skip over the superroot bit when the page is in the tag working set; we never risk reading from an unallocated page of tags. Property 3.6 implies that the page must be allocated if the page is in the tag working set.

The *Efficient Tagged Memory* scheme uses a trick to avoid some unnecessary writes: when we clear a leaf cacheline, we need not write the cleared cacheline back into the leaf table, and instead we can simply clear the root bit, as when we next read these tags, even if their values in the table are stale, we will encounter the cleared root bit first, allowing us to take a shortcut to returning zeroes. This works because we always read the root before the leaf.

However, the previously mentioned design choice I made in the Phoenix scheme's read algorithm means we cannot use the same trick when we clear a group of root bits, as we do not always read the superroot before the root. This means that when a group of 64 root bits becomes clear, we must write them all back as zeroes, as well as updating the superroot bit. If we do not do this, then when we skip over the superroot table during a read, we might end up following a stale root table entry down into a stale leaf table cacheline, reading out 1s when we should be reading 0s. This can be seen in the algorithm for writing.

The following is the algorithm for writing:

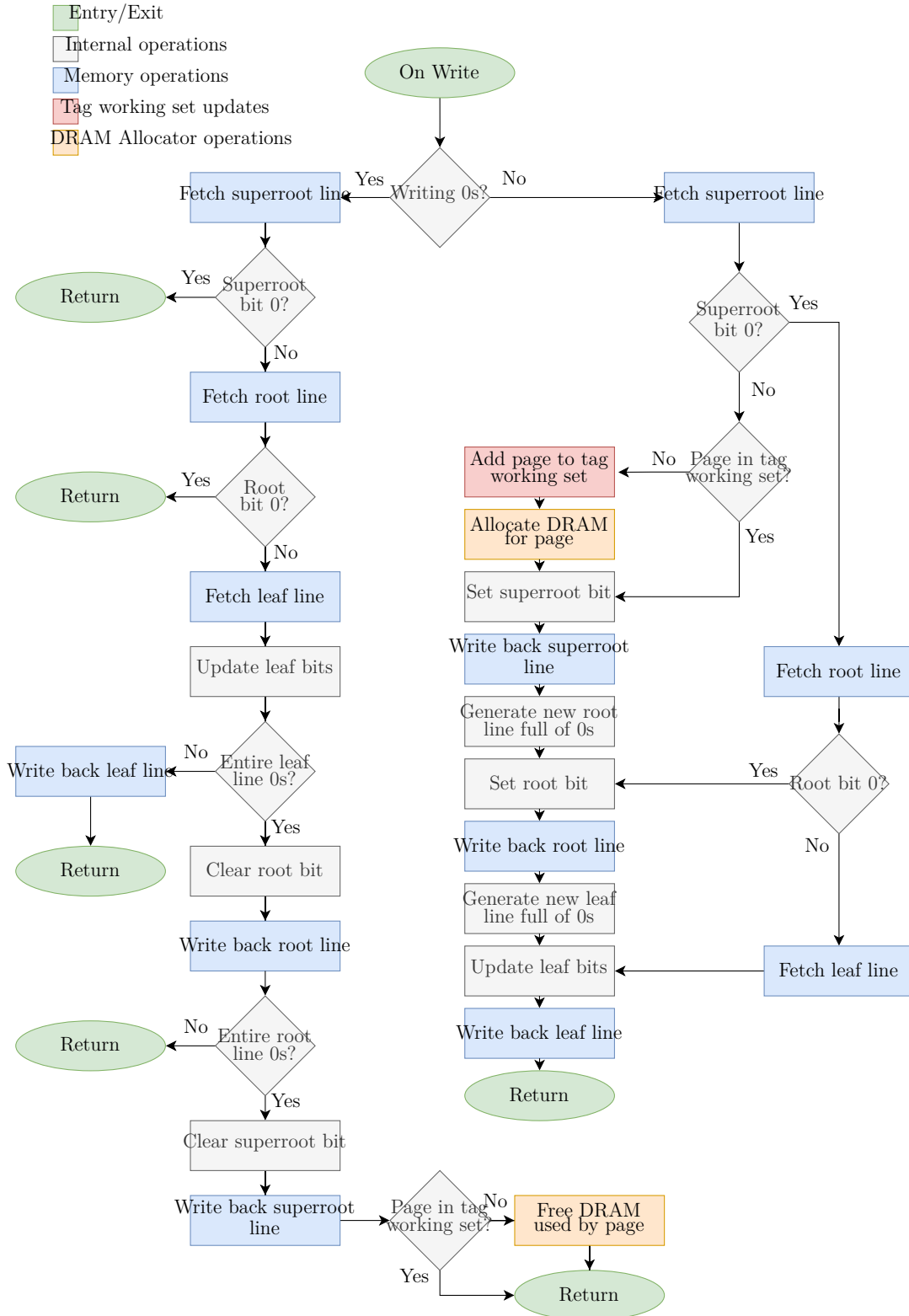


Figure 3.16: The Phoenix algorithm for writing tags. DRAM allocator operations are orange.

In the case where we update the tag working set and then immediately allocate a page in DRAM, there might be an opportunity to bypass the dynamic DRAM allocator. If the page we evict from the tag working set happens to contain no set tags, we would

free it. Instead, as we are about to allocate a page, we might instead be able to simply transfer the page from one superroot bit to another, perhaps by changing mappings in the appropriate table. Whether or not this would be possible would depend on the interface of the DRAM allocator, and what is exposed to the guest OSes.

3.5.4 The Implementation

`PhoenixSController.h` defines the Phoenix scheme's tag controller.

On constructing an instance, two interesting parameters must be passed:

`twSize` is the maximum size of the tag working set. A smaller set would mean that on average fewer DRAM pages would be allocated, resulting in the average space overhead of the tag table decreasing. Also, lookup times and area consumption of the actual physical tag controller would decrease. A larger tag working set would provide greater locality benefits, and perhaps less allocator thrashing.

`useTrueLRU` defines whether we are simulating an exact LRU replacement scheme for the tag working set, or an approximation of it, namely NRU. If this parameter is specified as `false`, the NRU scheme used will be a worst-case NRU. That is, the second-most-recently used page will be evicted. This way, we can get an approximate lower bound on the performance of an actual implementation of non-true LRU.

The tag working set itself is implemented as a `std::vector` of page indices. Whenever we access non-zero tags in a page, we move the index of that page to the back of the vector if it is already in the set, or add it to the back if it is not.

Whenever we add a page, we must check that the size of the vector has not exceeded `twSize`. If it has, we choose our victim. If `useTrueLRU` is true, we pick the index at the front of the vector. Otherwise, we pick the index just before the back.

To check if an index is in the tag working set, we simply do a linear pass over the vector.

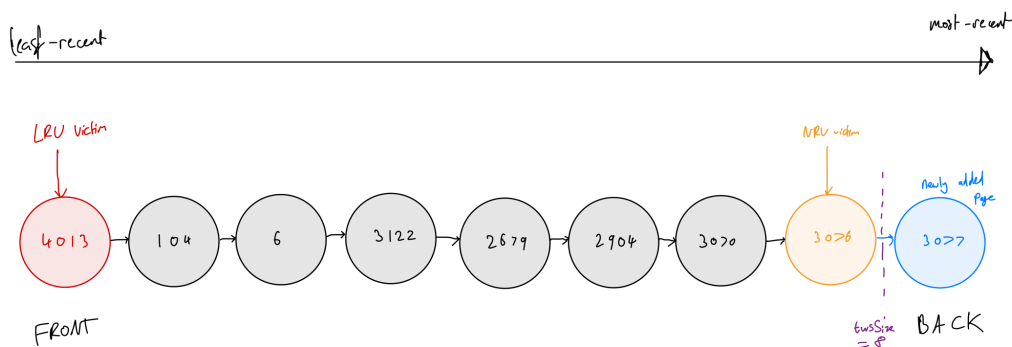


Figure 3.17: A visualisation of the implementation of the tag working set, and how the replacement policies choose their victims.

Realistically, the `twSize` will not be larger than a three-digit number, so the $O(n)$ cost that these operations have on the running time of the simulator is negligible.

We use a `std::unordered_set` to keep track of which pages are allocated in DRAM as the simulation progresses. The set is updated by the `DRAMAlloc` and `DRAMFree` methods, which are called by the `handleRead` and `handleWrite` algorithms. It is also initialised during `setup`.

After each LLC miss that the controller serves, it records how many pages are currently allocated in DRAM – i.e. the size of the *unordered_set*. These sizes are stored in a frequency list, implemented as a `std::unordered_map` which maps a count to the frequency of that count. `reportStats` uses this frequency list to report some metrics at the end of the simulation.

3.6 Repository Overview

—

Chapter 4

Evaluation

In this chapter, I first present statistics provided by my tag controller simulator, and use them to evaluate the two currently used tag controller schemes, addressing Question 1, and also my novel tag controller scheme, addressing Question 2. I then summarise and reflect on the utility of the tag controller simulator as a tool for fostering innovation in the field of computer architecture.

4.1 DRAM Access Latency

The following data was gathered by simulating how the tag controllers from the basic scheme and from the Morello Project scheme handled 50 million LLC misses from several different workloads. The workloads were run on a simulated CHERI processor with 2GiB of memory.

By default, I simulated a 4-way 32KiB tag cache, with no prefetcher. Some plots show how varying one of these parameters affects the performance of the cache.

4.1.1 Flat Table Controllers

The statistics for the Morello Project scheme were obtained by summing the values from the two simulations of the two caches, as equations 4.1 and 4.2 describe.

$$\begin{aligned} \text{Total number of tag cache accesses} = & \text{Number of accesses to zero cache} \\ & + \text{Number of accesses to non-zero cache} \end{aligned} \tag{4.1}$$

$$\begin{aligned} \text{Total number of tag cache misses} = & \text{Number of zero cache misses} \\ & + \text{Number of non-zero cache misses} \end{aligned} \tag{4.2}$$

4.1.1.1 Performance

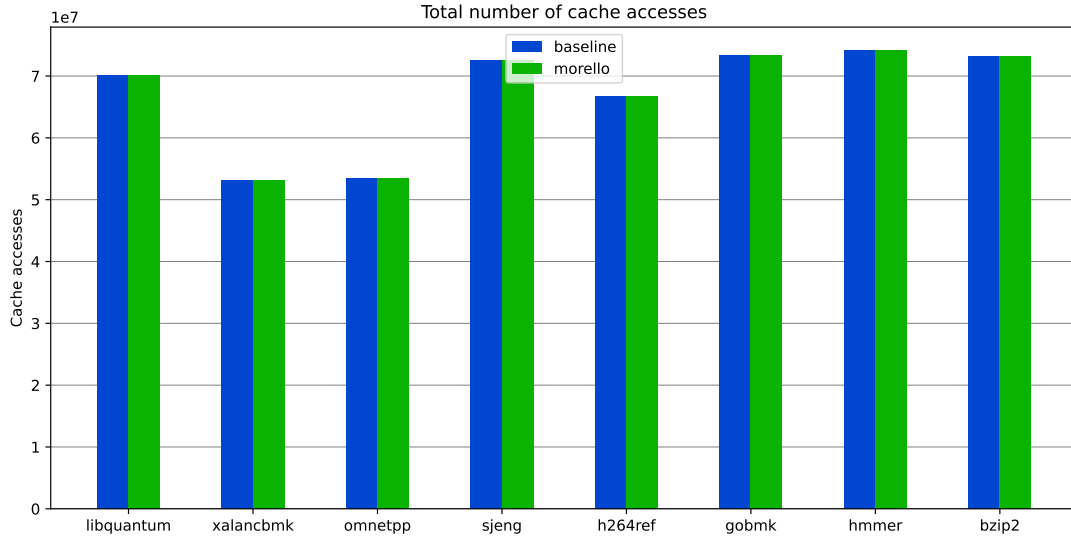


Figure 4.1: Tag cache access frequencies of the flat table controllers on various workloads.

Figure 4.1 shows how both the Basic controller and the Morello Project controller always access the tag cache the same number of times. This is what we expect, as despite the cache arrangement differing, their algorithms (ignoring the pseudo-memory access operations of the Morello Project controller) are fundamentally the same if we treat the two caches of the Morello Project scheme as a single merged black-box cache.

The figure also shows how the number of tag cache accesses each controller makes varies for different workloads. This is because different workloads have a different split between read operations and write operations. In these schemes, reads require one access to the tag cache, but writes require two – one to fetch the cacheline, and one to write it back after it the target tags have been updated.

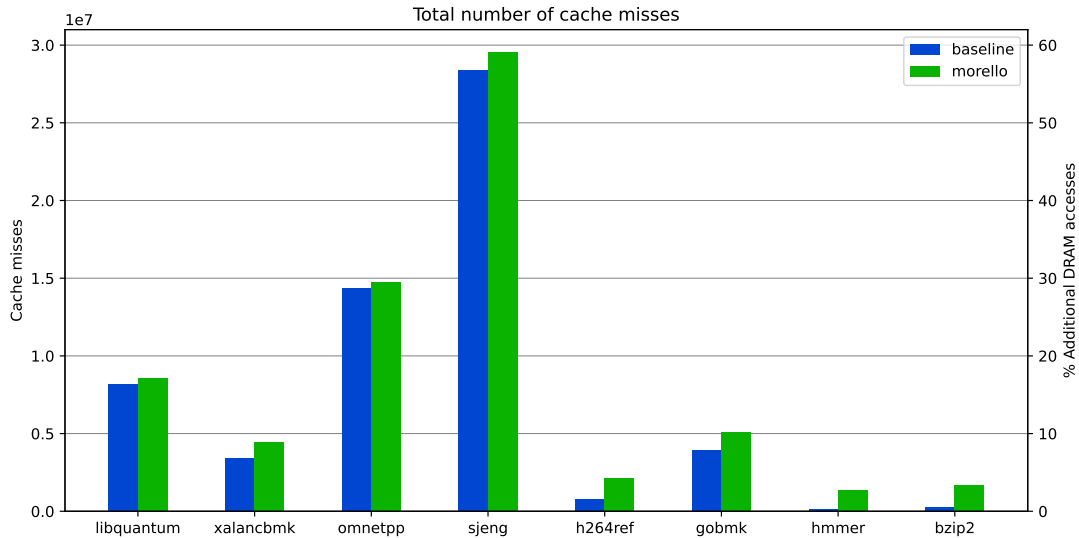


Figure 4.2: Tag cache miss frequencies of the flat table controllers on various workloads.

Figure 4.2 shows how many misses the tag caches of the two flat table controllers experience. This in turn tells us for how many of the 50 million LLC misses we must perform an additional DRAM access to retrieve the tag data, and thus we can get an idea of how much DRAM latency overhead the schemes bring.

The tag controller serves requests from the last-level cache. These requests exhibit relatively little spatial and temporal locality, as all of the accesses that do exhibit such locality would very likely have hit in the other caches further up the hierarchy; only the least spatially and temporally local accesses make it down to the below last-level cache. Thus, we would expect the miss rate of the tag cache in the basic scheme to be roughly equal to that of a last-level data cache. For the SPEC benchmarks, the LLC miss rate is around 40-60% [ccc cite]. We see in figure 4.2 that in the case of the basic controller, we observe miss rates of up to 57%, and thus we would have to access DRAM this much more often.

These additional requests to the tag table in DRAM could be pipelined with the value data DRAM accesses, so we would expect latency to increase by an overall factor of up to $0.43 + 0.57 \times 1.1 = 1.057$.

4.1.1.2 Varying Capacity

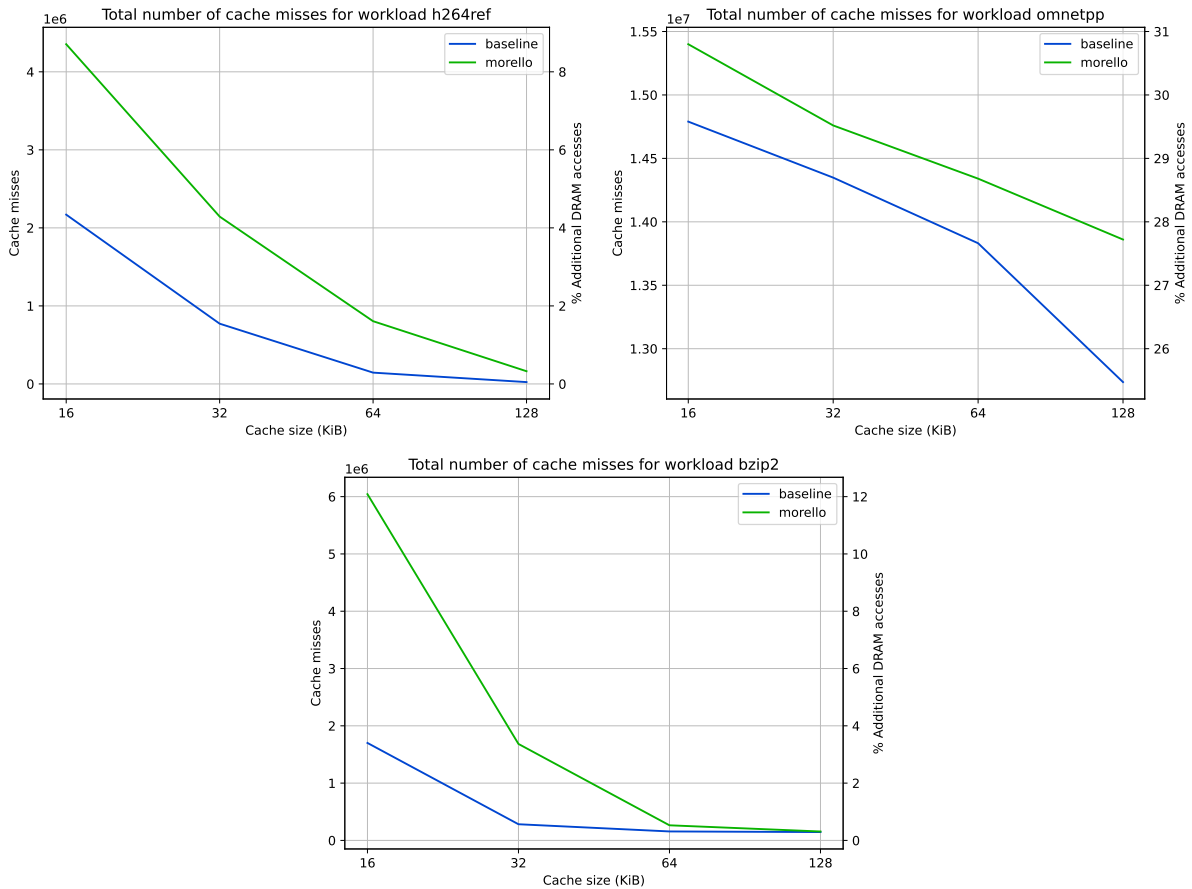


Figure 4.3: How tag cache size affects miss frequencies of the flat table controllers.

As we increase the capacity of the cache, we expect its hit rate to increase. We see exactly this in figure 4.3. The benefits of increasing the cache tend to decay exponentially, although there is an interesting exception to this trend when going from a 64KiB cache to a 128KiB cache when running the omnetpp workload. It might be the case that the tags of the data contained within working set of the omnetpp workload do not quite fit into 64KiB, but do fit into 128KiB. Thus, bridging the gap to an 128KiB cache means we begin to avoid many conflict misses that were previously occurring in a 64KiB cache.

4.1.1.3 Varying Ways

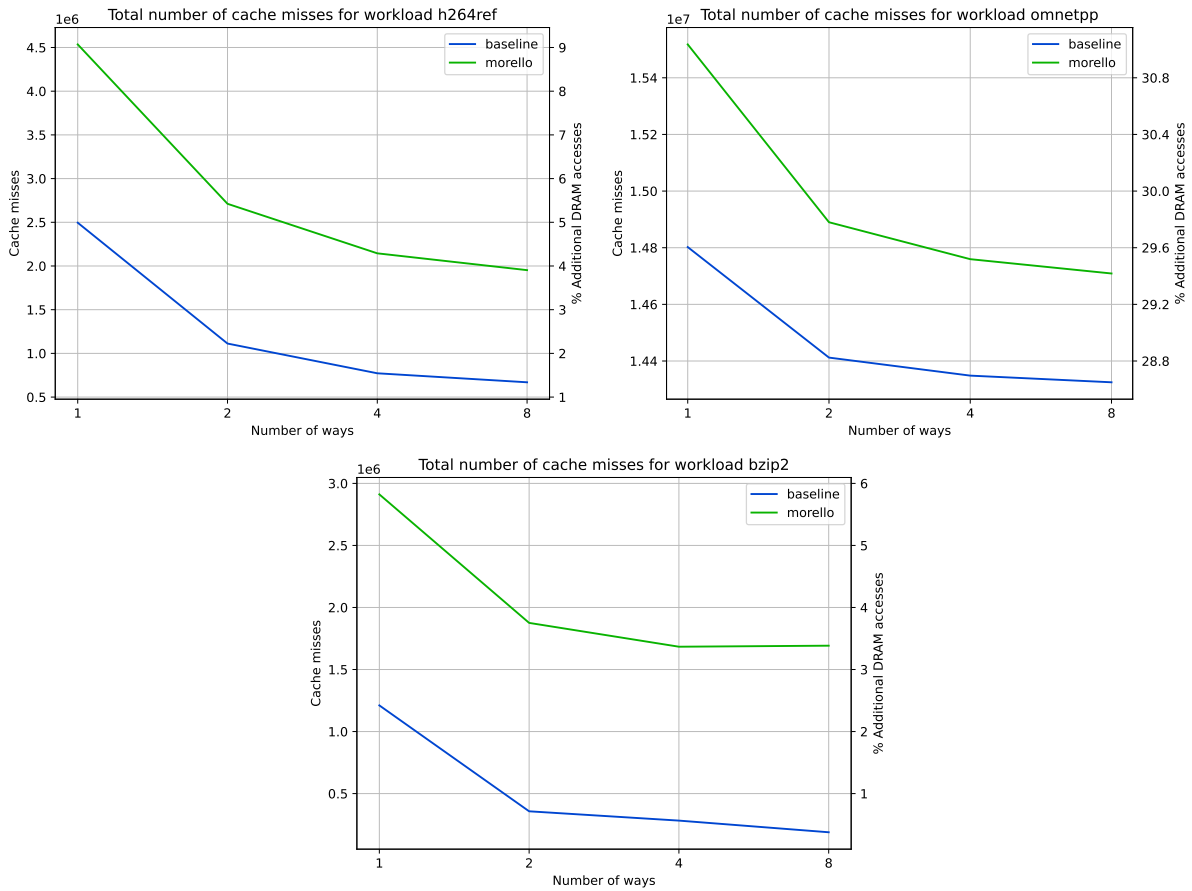


Figure 4.4: How the number of ways in the tag cache affects miss frequencies of the flat table controllers.

As we increase the number of ways in the cache whilst holding the capacity constant, we expect to see some improvement in hit ratio, but with diminishing returns. The plots in figure 4.4 show exactly this.

4.1.1.4 Adding a Tag Cache Prefetcher

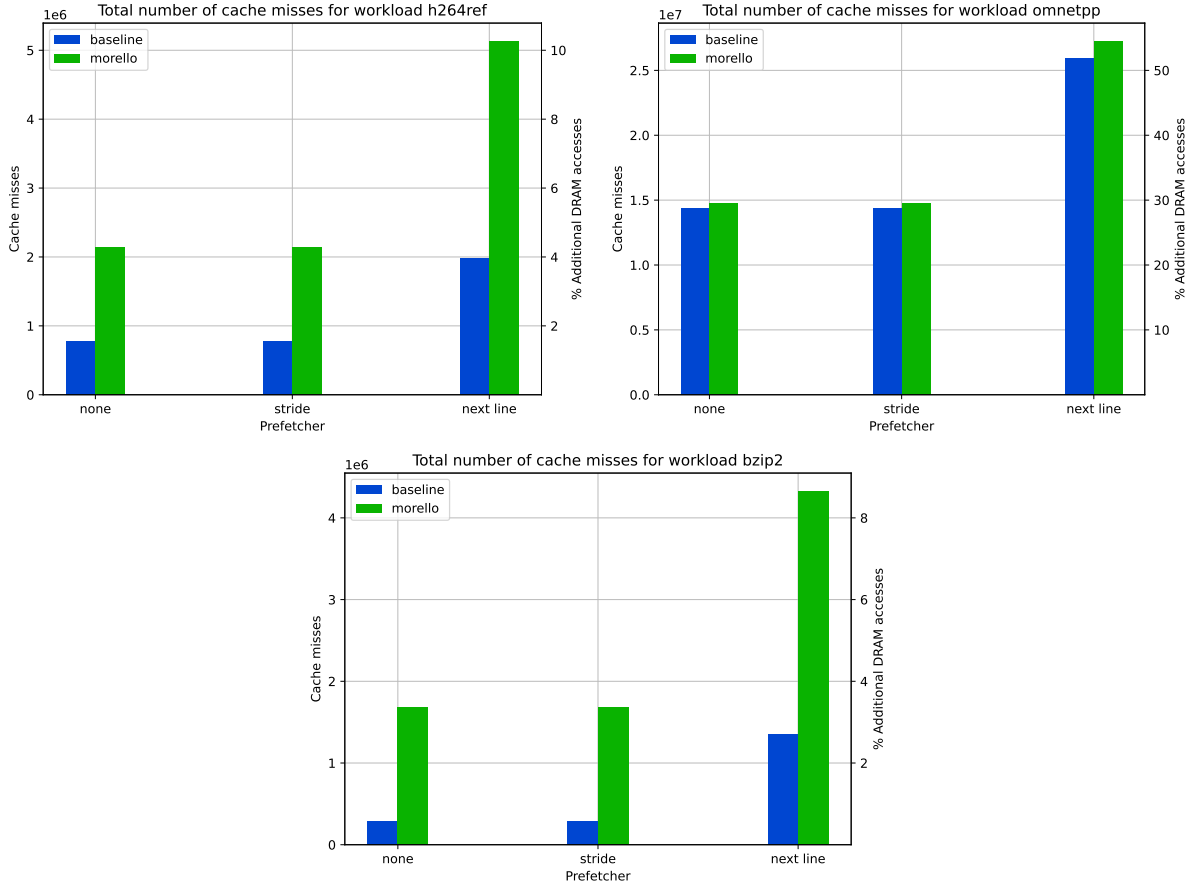


Figure 4.5: How adding a prefetcher to the tag cache affects miss frequencies of the flat table controllers.

Prefetchers aim to avoid compulsory misses by assuming a large degree of spatial locality in the access pattern. However, as I described before, the tag cache is effectively a last-level cache, meaning there is very little locality to exploit at this level. We would thus expect to find that prefetchers, at least simple ones, cannot help much to reduce misses in the tag cache.

I enabled two of ChampSim’s provided prefetchers and simulated the benchmarks. We can see in figure 4.5 that the stride prefetcher hardly changed performance, whilst the next line prefetcher made performance significantly worse. The worsening in performance is caused by the prefetcher fetching lines that cause useful lines – ones that the controller was going to access again – to be evicted before that subsequent access, when they otherwise would not have been.

Using simple prefetchers does not seem like a viable option in improving the performance of tagged memory architectures. ChampSim provides two other built-in prefetchers, “spp_dev” and “va_ampm_lite”, however I was unable to get these to work.

4.1.2 Hierarchical Schemes

4.1.2.1 Performance

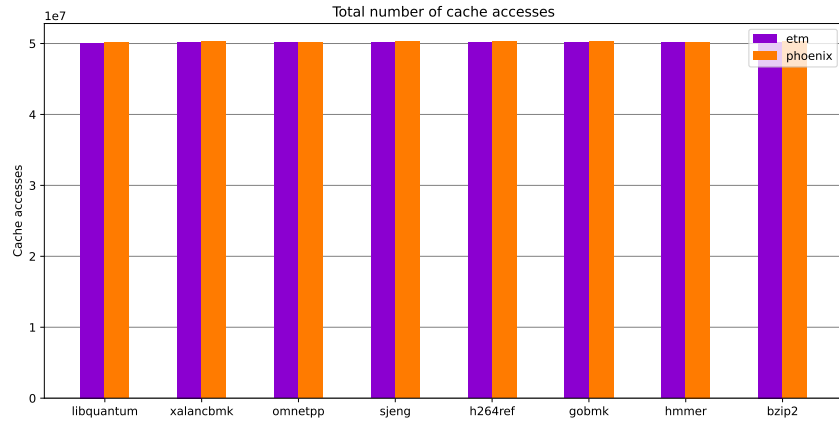


Figure 4.6: Tag cache access frequencies of the hierarchical table controllers on various workloads.

When a hierarchical tag controller is handling an LLC request, if it finds that the root or superroot bit is 1, the controller must traverse one level deeper into the hierarchical table. This requires an additional tag cache access. If this happens a lot, we would expect to see that these controllers perform many more than 50 million tag cache accesses.

Figure 4.6 shows that whilst handling the 50 million LLC misses, the two hierarchical tag controllers hardly made any more than 50 million accesses. This is a testament to the efficiency of the hierarchical table – only rarely must we traverse deeper than the highest level.

This fact is particularly promising for the Phoenix scheme. If it were the case that the optimal scope of the top-level bits was that of the *Efficient Tagged Memory* scheme (one leaf cacheline per root bit), then we would expect to see the number of cache accesses rise when we add a third layer to the table with a larger scope. However, we do not see a significant increase in the number of accesses from the Phoenix controller, indicating that, at least for these workloads, it is beneficial to have a larger top-level scope.

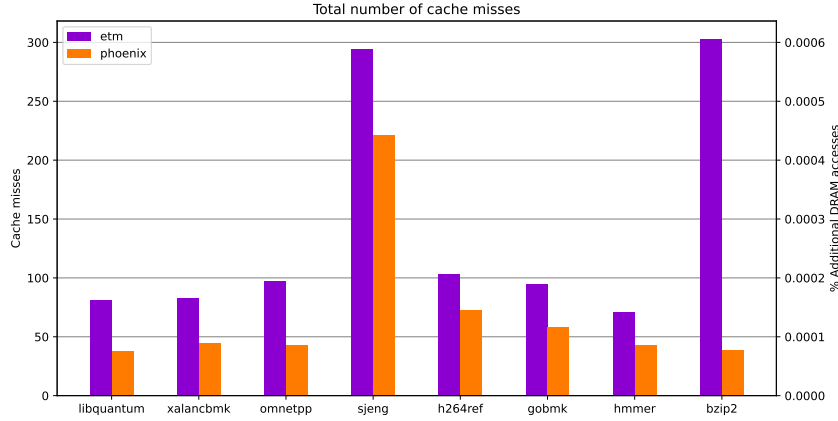


Figure 4.7: Tag cache miss frequencies of the hierarchical table controllers on various workloads.

The tag cache miss rates are remarkably low for these schemes. We see no more than 303 tag cache misses caused by the handling of all 50 million LLC misses across all workloads. A potential explanation for the low miss rates might lie in the assumption scheme used to make educated guesses at the initial values of the tags in the tag table. The assumption scheme has some bias towards assuming tags are 0, as when the tags were initially accessed by a regular load or store, we assume that the data being accessed was non-capability data, though in reality it might have been capability data. This bias leads to the simulated tag table being sparser with tags than it was in reality. The hierarchical schemes are particularly good with sparse tables as they can store zeroes more compactly in the cache, thus their performance benefits from this bias.

The Phoenix tag controller appears to always outperform the *Efficient Tagged Memory* controller, often by a significant margin.

Interestingly, in the case of the bzip2 the difference in performance is incredibly large – the Phoenix tag cache sees only about 13% of the number of misses that the *Efficient Tagged Memory* scheme does.

This might be because this workload has many unset tags, meaning that more there are more superroot bits set to zero resident in the cache. This would increase the effective scope of the cache, which in turn increases the hit rate.

The exact resulting DRAM access latency overhead depends on which accesses missed. Suppose two cache misses occurred during the handling of a single LLC miss. As the accesses in one pass of the tag controller algorithm contain a data dependency, one could not be pipelined with the other. Whilst the former DRAM tag table access could be pipelined with the DRAM data access, the latter DRAM tag table access would have had to wait for the former to complete, resulting in an overhead factor of about 2.1. However, if the two cache misses occurred in the handling of two different LLC misses, then the accesses would be in two different passes of the algorithm, and the two resulting DRAM tag table accesses could both be pipelined with their respective DRAM data accesses, resulting in an overhead factor of about 1.1.

As we do not know exactly which accesses missed, we can only state bounds for the

resulting DRAM access latency overhead. In the best case, all of these misses occurred during handling of independent LLC misses, resulting in no data dependencies. The table in figure 4.8 explains the worst-case number of dependent misses the controllers might make during a single pass of their algorithms.

Scheme	Number	Explanation
ETM	2	When writing 1s to a leaf line whose root bit is 0, the root and leaf lines were not in the tag cache.
Phoenix	3	When writing 1s to a page whose superroot bit is 0, the superroot, root and leaf lines were not in the tag cache.

Figure 4.8: The worst-case number of dependent memory accesses of the hierarchical table tag controller algorithms.

Using these values, we can calculate the best- and worst-case DRAM access latency overheads for the different schemes. In the best case, we see 10% overhead. For the worst case, if m is the miss rate and c is the number of dependent accesses, the formula for the percentage DRAM access latency overhead is as follows:

$$\% \text{ Overhead} = m \times (0.1 + (c - 1)) \times 100 \quad (4.3)$$

The plot in figure 4.9 uses this formula to calculate the bounds for DRAM access latency overhead. Even in the worst cases, the overhead is extremely low.

The plot also reveals that the worst case additional DRAM access latency overhead for the Phoenix scheme is sometimes worse than that of the ETM scheme, due to its additional potential dependent access.

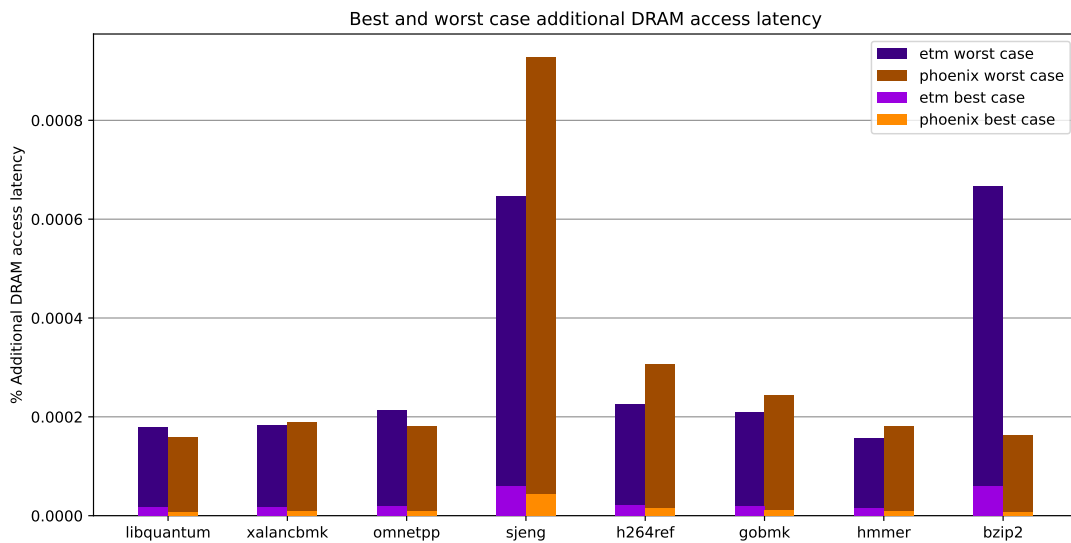


Figure 4.9: Best- and worst-case DRAM access latency overhead for the hierarchical table tag controllers.

4.1.2.2 Varying Capacity

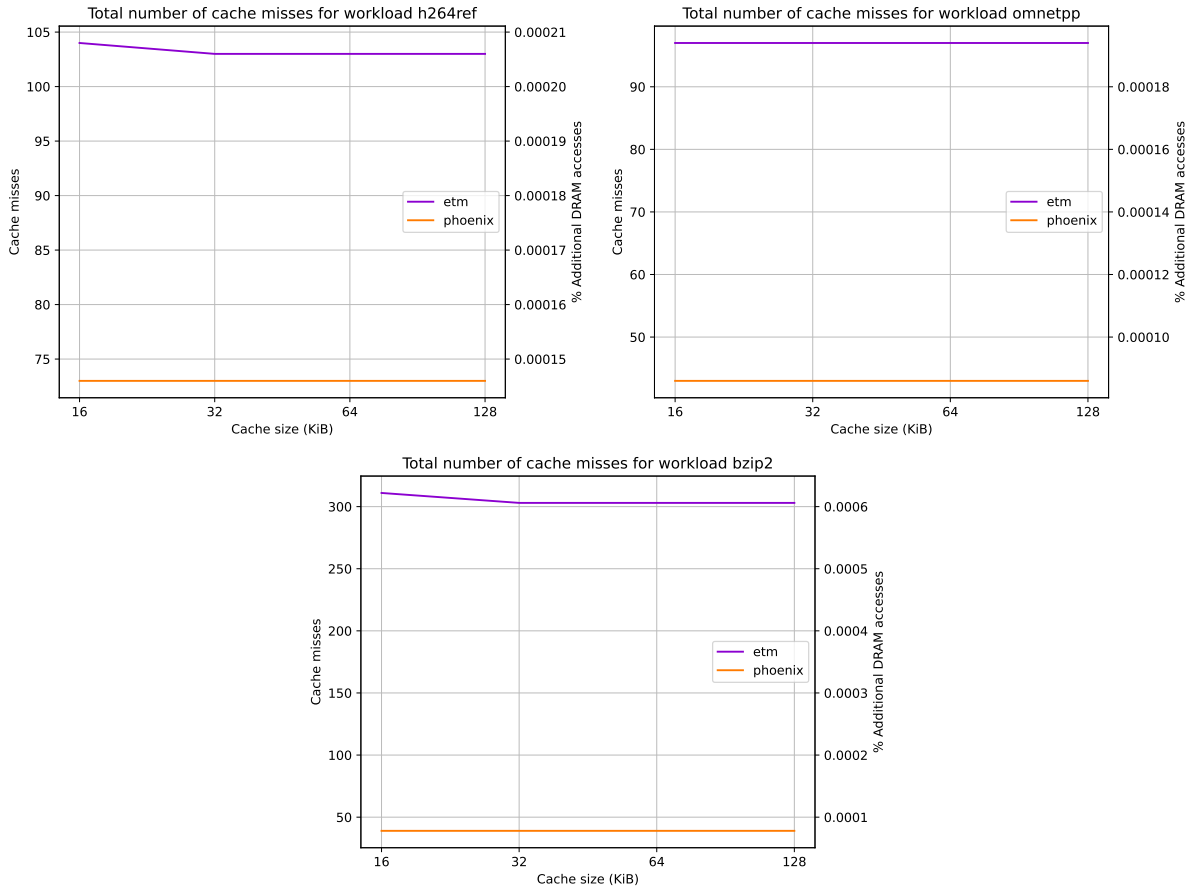


Figure 4.10: How tag cache size affects miss frequencies of the hierarchical table controllers.

Here, we notice that increasing the capacity of *Efficient Tagged Memory*'s tag controller's cache only really improves performance when going from 16KiB to 32KiB. In the case of the Phoenix controller, increasing the size never helps.

This is because the working set of tags is already essentially covered in full, thanks to the large scope of the root and superroot bits of these schemes. This is why these caches see such low miss rates.

4.1.2.3 Varying Ways

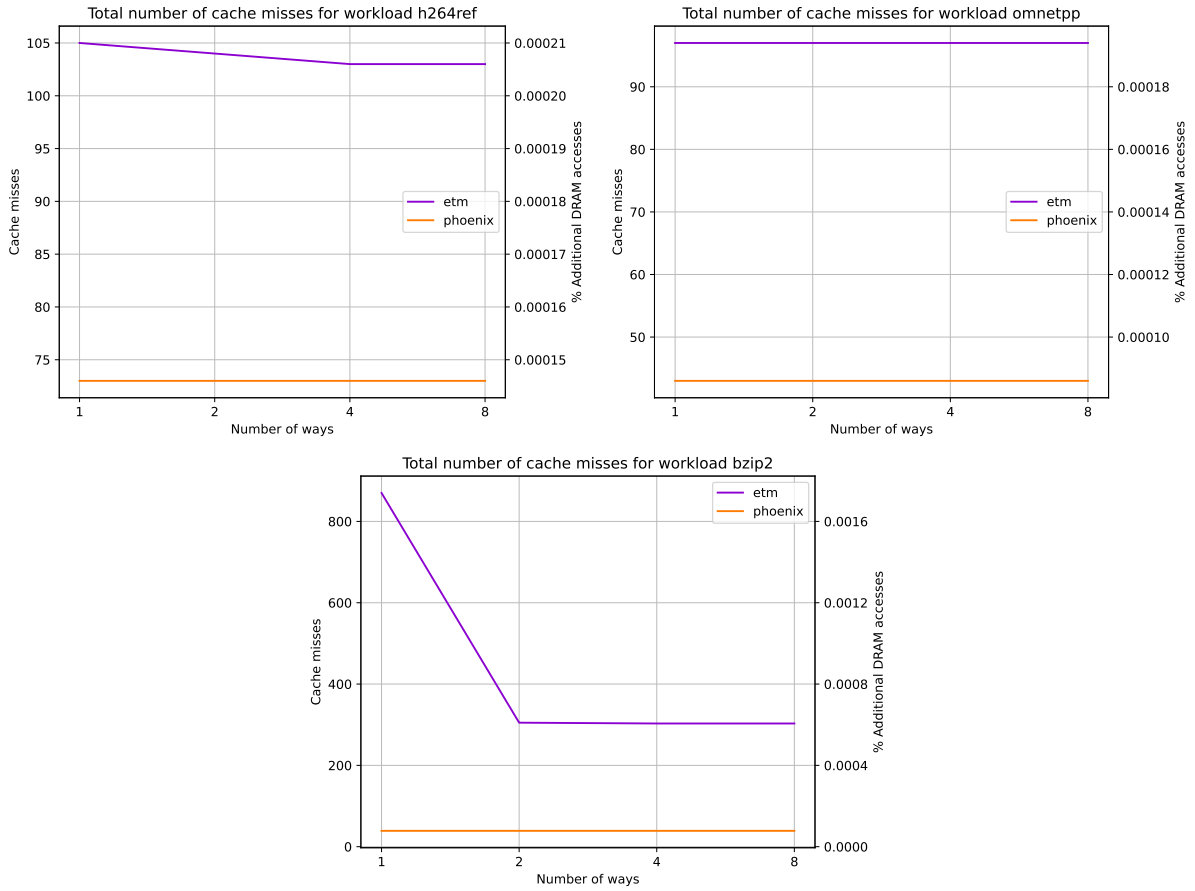


Figure 4.11: How the number of ways in the tag cache affects miss frequencies of the hierarchical table controllers.

Increasing the number of ways can improve cache hit rate by eliminating conflict misses; specifically those that were caused by cachelines in the same working set with an equal index evicting each other. If the stride size of an algorithm results in many distinct cachelines with equal indices being frequently accessed, we would expect to see lots of conflict misses when the number of ways is small.

The tag table has a scale of 1:128 with the value portion of DRAM. Thus, for a tag cache with 128 sets and 64B cachelines, the algorithm would have to have a working set that spans across $128 \times 128 \times 64 = 1\text{MiB}$ to experience a conflict. This is unlikely, but not out of the question. However, with a root table, which has a scale of 1:65536, the working set must span $65536 \times 128 \times 64 = 512\text{MiB}$ of data, which is exceptional. With a superroot table, the trend continues.

From Figure 4.11 we can see that increasing the number of ways beyond 2 hardly helps to improve tag cache hit rate. My theory is that due to the above reasoning, we find that increasing the number of ways in the cache when using a hierarchical scheme hardly helps as we already mitigate conflict misses thanks to compressing the zero tags into a smaller cacheline scope, thus decreasing the chance of a conflict miss.

4.1.2.4 Adding a Tag Cache Prefetcher

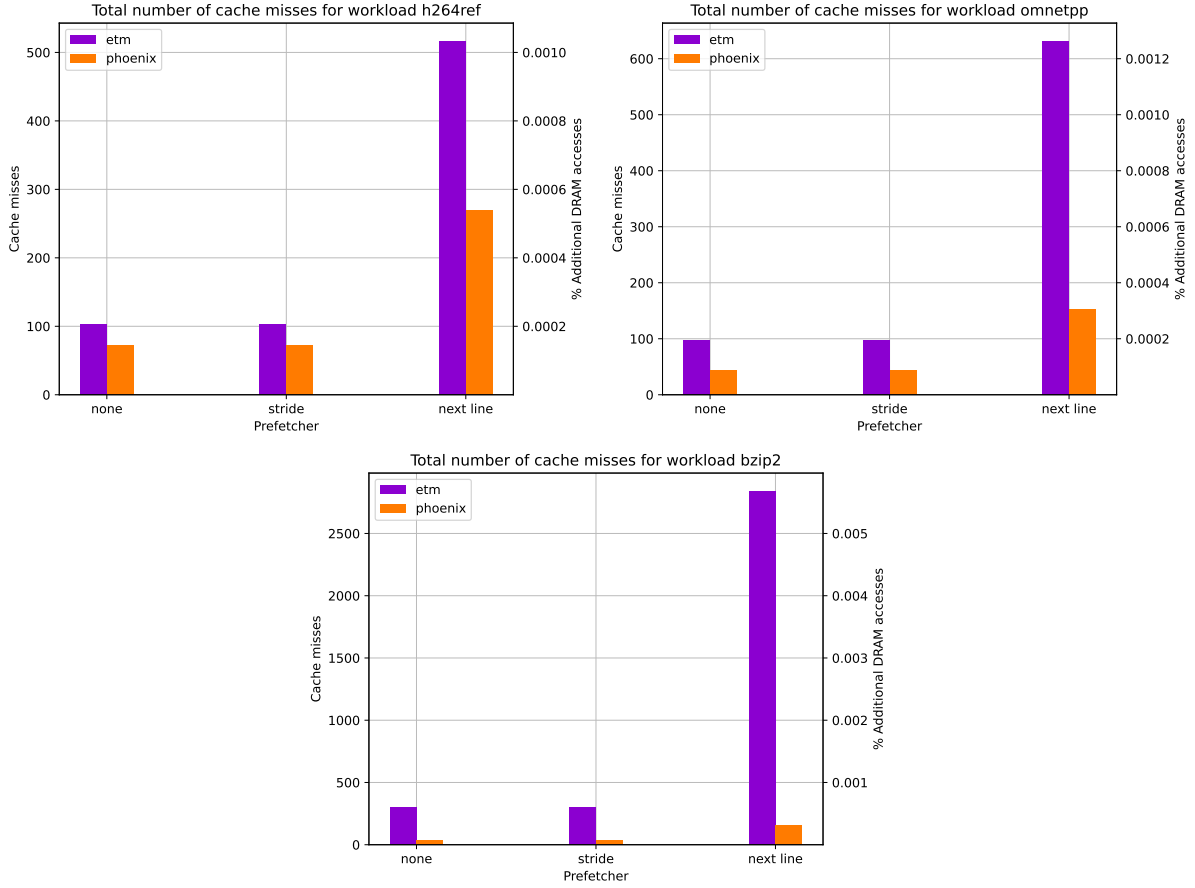


Figure 4.12: How adding a prefetcher to the tag cache affects miss frequencies of the hierarchical table controllers.

Once again, we find that adding simple prefetchers either has no effect, or makes performance worse.

4.2 DRAM Space Consumption

4.2.1 Theoretical Limits of Tag Table Compression

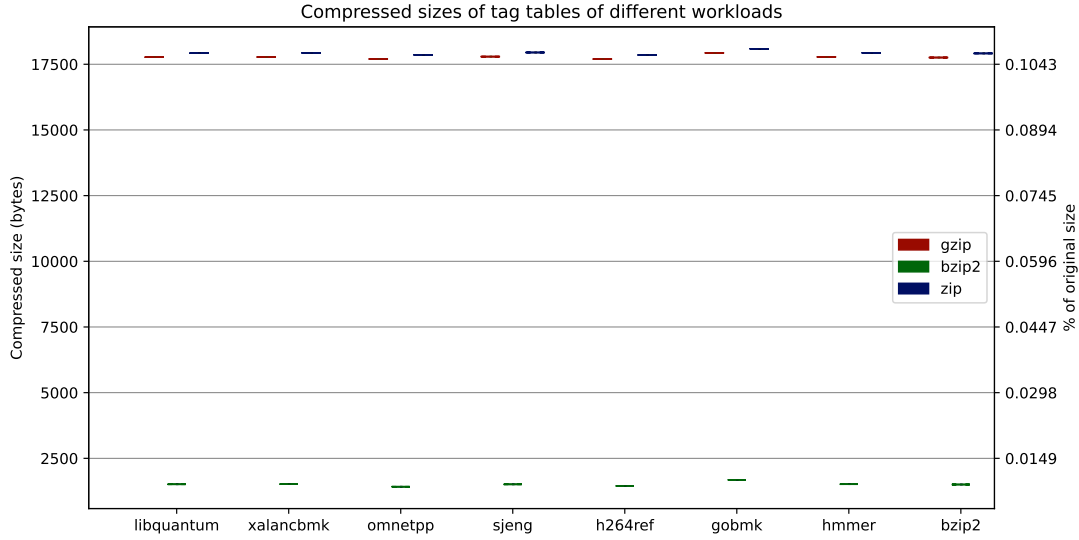


Figure 4.13: Various compression algorithms applied to the tag tables of various workloads.

4.2.2 Dynamic DRAM Allocation

When attempting to compress information, it is useful to know the limit of compression – the minimum number of bits required to losslessly decompress that information. I attempted to get an idea of the limits of compression of the tag table by dumping the state of a flat tag table to a file, and running three different compression algorithms on that file. The box plot in figure 4.13 was generated by compressing the state of the tag table at 20 different points during the simulation of the basic tag controller handling 50 million LLC misses. We can see that the table can be compressed to between about 0.01% and 0.1% of its original size. Whilst these are not absolute lower bounds, we know that they are possible levels of compression that a tag controller scheme might be able to achieve, albeit likely very ambitious to actually do so whilst maintaining reasonable runtime performance.

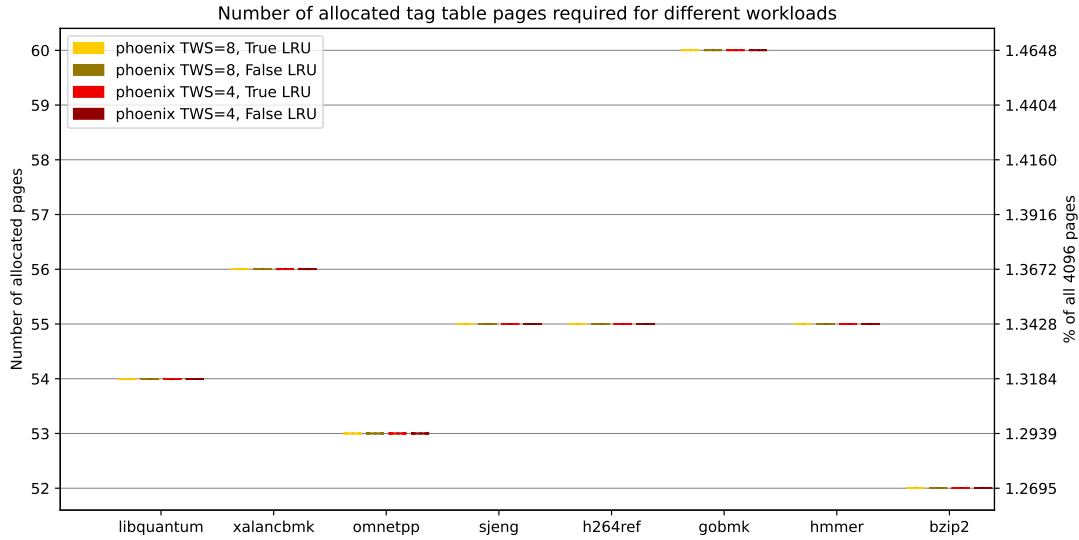


Figure 4.14: The space consumption of the tag table when using the Phoenix scheme’s on-demand DRAM allocation.

Figure 4.14 shows how many pages were kept allocated during the handling of the LLC misses, and also this number as a percentage of the entire leaf table. We can see that the compressed sizes varied between about 1.27% and 1.46%, which is between 12.7 and 146 times larger than what was achieved by the compression algorithms. Whilst there may still be lots of potential for compression, the fact that over 99% of the space consumed by the leaf table could be freed is noteworthy.

We can calculate the physical space consumed by the leaf tables by multiplying the number of pages by the size of a page, 4096B. The DRAM used by the root and superroot tables however must always remain fully allocated. With a 2GiB memory, they consume 5KiB and 512B respectively, so we must add 5.5KiB on top of these values to get the total DRAM space consumption overhead.

The results show that with the help of dynamic DRAM allocation, the leaf table, which was originally 4MiB in size, only needs to consume between 208 and 240KiB for these workloads to run under the Phoenix scheme, leaving the total additional DRAM space overhead at between 213.5 and 245.5KiB.

4.3 Evaluation of the Simulator as a Tool

The tag controller simulator provides a framework that allows an architect to implement an algorithm for a tag controller, and evaluate it. Specifically, when paired with ChampSim, it can report the following metrics for a given workload:

- tag cache access frequency (as a total, and for reads and writes separately)
- tag cache hit frequency (as a total, and for reads and writes separately)
- tag cache miss frequency (as a total, and for reads and writes separately)
- the number of tag cacheline prefetches made (if a prefetcher is enabled)

- the number of useful tag cacheline prefetches (if a prefetcher is enabled)
- the number of useless tag cacheline prefetches (if a prefetcher is enabled)

From the first two metrics, the hit ratio of the tag cache can be calculated.

On top of this, the accompanying visualisation tool allows an architect to visually inspect the state of the tag table and tag cache at points in the simulated execution. The tool allows visualisation for all of the three schemes described in the Preparation chapter, and it could be extended to visualise other schemes.

Initially, the success criteria of this project were written with the implication that the tag controller simulator alone, as a single program, would be responsible for producing all tag cache metrics. However, after some consideration I decided to use an existing cache simulator, ChampSim, to simulate the cache, and to have the tag controller simulator interface with ChampSim via its output trace file. When combined, the two programs provide a single tool that does indeed provide all of the necessary outputs, given the promised inputs, to meet all success criteria. An example Python script I wrote that combines the two programs by running one after the other can be found at `/CHERI-Tag-Controller/evaluation/eval-basic.py`.

Additionally, the simulation tool successfully reproduced the results found in *Efficient Tagged Memory*. By using the visualisation tool, I was able to easily investigate the state of the hierarchical scheme, and with this insight I was able to devise my own tag controller scheme.

The framework allowed me to implement my novel scheme with ease – I was only required to provide implementations for three class methods after extending the `Controller` class. I was then able to verify my novel scheme with the build-in verification program, and I could then evaluate it by simply providing the simulator with a workload.

The promising results of the Phoenix scheme suggest that the tool has genuine potential to accelerate innovation in tag controller design.

There are however some drawbacks to the tool:

- Several interesting parameters, such as the memory size of the simulated workload and cacheline size, cannot be simply changed – the values of 2GiB and 64B respectively are currently hard-coded into the tool.
- The visualisation tool is not straight forward to extend – unlike the tag controller simulator, there is not a programmer-friendly framework.
- The images that the visualisation tool are very large (around 100 Megapixels), and thus require non-standard image viewers, such as `vipsdisp` [9], to view them.

Chapter 5

Conclusions

Conclusions provide a clear and thorough summary, with interesting future work suggested. Thoughtful personal reflection on the lessons learned.

5.1 Project Successes

TALK ABOUT THE 2 QUESTIONS!!! able to reproduce ETM's results

5.2 Future Work

keep this brief The

5.3 Reflection and Lessons Learned

Bibliography

- [1] Alexandre Joannou et al. “Efficient tagged memory”. In: *2017 IEEE International Conference on Computer Design (ICCD)*. IEEE. 2017, pp. 641–648.
- [2] Brian E. Clark and Michael J. Corrigan. “Application System/400 performance characteristics”. In: *IBM Systems Journal* 28.3 (1989), pp. 407–423.
- [3] Jedidiah R Crandall and Frederic T Chong. “Minos: Control data attack prevention orthogonal to memory model”. In: *37th International Symposium on Microarchitecture (MICRO-37’04)*. IEEE. 2004, pp. 221–232.
- [4] Joseph L Greathouse et al. “A case for unlimited watchpoints”. In: *ACM SIGPLAN Notices* 47.4 (2012), pp. 159–172.
- [5] G Edward Suh et al. “Secure program execution via dynamic information flow tracking”. In: *ACM Sigplan Notices* 39.11 (2004), pp. 85–96.
- [6] <https://www.cl.cam.ac.uk/research/security/ctsrd/cheri/>.
- [7] <https://www.morello-project.org/>.
- [8] <https://github.com/ChampSim/>.
- [9] <https://github.com/jcupitt/vipsdisp/>.

Appendix A

Dummy

Appendix B

Project Proposal

Next Generation CHERI Tag Controller

Kofi Wilkinson, Homerton College

Project Originator: Dr Jonathan Woodruff

16th October 2023

Project Supervisor: Dr Jonathan Woodruff

Director of Studies: Dr John Fawcett

Project Checkers: Prof. Alan Blackwell & Prof. Srinivasan Keshav

Introduction

Background

The paper *Efficient Tagged Memory* [1] describes how tagged memory architectures have been explored in academia and employed in industry many times with various different focuses throughout the history of computer design. Through both academic rigour and market testing, they have been proven to provide system-wide security properties, such as hardware capabilities [2], pointer integrity [3], watchpoints [4], and information-flow tracking [5]. The paper continues to explain how single-bit tag (SBT) shadowspaces are sufficient to provide these properties, as one bit is enough to distinguish between data that has been manipulated by untrusted code, and data that retains the integrity of only being written by a program running through a trusted path of execution, and with this we can define as many new hardware-enforced types as we might need.

Section III of the paper describes four different approaches to implementing tagged memory. A direct quote of this list is as follows:

- *width* – Store tag bits in a wider physical memory, e.g., 65-bit words.
- *in-page* – Store tag bits in borrowed bytes of DRAM pages, slightly shrinking each DRAM page and skewing the DRAM address mapping.
- *dedicated* – Store tag bits in a dedicated memory.

- *table* – Store tag bits in a table in DRAM.

As the paper then explains, the *width* option is conceptually the simplest as its only unique requirement is to widen the system memory and buses. This approach has been taken before, e.g. by the Burroughs B6000 [6], however, power-of-two memory systems dominate today’s industry, making this approach infeasible for an SBT architecture.

The *in-page* option leads to a fragmented tag shadowspace, limiting both tag cacheline size and spatial locality benefits. The *dedicated* option would require its own memory subsystem, complete with a dedicated interface and discrete chips, which brings too much design, area and power overhead for most applications.

This leaves the *table* option, and it is this that the rest of the paper concerns itself with optimising due to its practicality from having relatively little overhead. The overhead can be reduced by making the tag table more cacheable to decrease the frequency of suffering DRAM latency penalties. The tag table memory accesses and tag caching are all handled by a hardware tag controller, which determines the algorithm by which tags are read and written during execution.

Architectures that employ a tag table will statically allocate a region of DRAM at boot-time to act as the tag cache. This of course results in a static memory cost, which is typically acceptable for client applications, but unacceptable for server/cloud applications, according to feedback from industry. Different approaches of caching tags have been explored, which aim to reduce the DRAM tag table access frequency, but they do not address the constant DRAM space consumption overhead of the tag table itself. *Efficient Tagged Memory* details a hierarchical lookup scheme that compresses tags within the tag cache, thus storing more information about the tag table within the tag cache, further increasing tag cache hit rate, but it in fact has the effect of increasing the constant DRAM space consumption slightly due to the addition of a first-layer.

Project Focus

An idea that has not yet been explored comprehensively in literature is dynamic allocation of shadowspace for the tag table. Doing this would allow the following techniques to be used to reduce the tag table’s DRAM space consumption:

- We could request only as much tag memory as is required for the current workload and free it back out to the rest of the system once it is no longer required.
- We could compress the tag table in DRAM, and then free the extra saved space.

One possibility is dynamic allocation of physical DRAM to different guest OSs running on one machine, managed by a hypervisor. How such a system might work is not the focus of my project, but mechanisms for dynamic allocation exist [7].

If a substantial amount of space can be saved by the two mentioned techniques, tag table implementations of memory, and subsequently tagged memory architectures, would take a significant step towards becoming viable for use in server/cloud applications, meaning that the aforementioned benefits they have provided to client applications can also be brought to these industries.

Starting Point

- I have read the paper *Efficient Tagged Memory*.
- I completed the Cambridge SystemVerilog Tutor [8] and performed the mandatory tasks 1 and 5 for the Part IB tick, providing me with some HDL experience with SystemVerilog. I also completed the optional task 2a, providing me with an introduction to HDL simulation with ModelSim.

- Over the summer of 2023 I interned at Arm in the Morello Kernel Team, a team working to port the Linux kernel to Arm’s Morello architecture, which is based on the CHERI model. This gave me a chance to get more familiar with the CHERI model, and provided me with an understanding of how a CHERI tagged memory architecture might structure its pointers, and how tag shadowspace might be laid out and accessed in DRAM.

Substance and Structure

My project has several distinct but related objectives. The order in which I have listed the objectives is the order in which it makes sense to tackle them.

Throughout this project I will be focusing on SBT architectures due to their preference in industry and their simplicity. I will be focusing particularly on the CHERI architecture, as this is a speciality of the university and of my supervisor, and it is also the tagged memory architecture I am most familiar with, thanks to my aforementioned summer internship. Much of my work and findings however will be transferable to other tagged memory architectures.

Tag Controller Simulator and Algorithm Analysis (Core)

The **core** objective for my project is to develop a tag controller simulator that can be used to evaluate two existing algorithms for tag table storage. Given a workload, it should be able to determine the state of the tag cache and tag table at any point during program execution, the tag cache’s hit ratio, the DRAM tag table access frequency and the overall execution stall time due to DRAM access latency.

I intend to develop the simulator in C++ for the following reasons:

- I am familiar with it from the IB Programming in C and C++ course and from contributing to the development of my IB group project in C++.
- It is a highly performant language which is important when parsing and simulating very large memory access traces.
- The traces are in a format which can be parsed very quickly and easily by existing C++ libraries.
- Lots of existing cache simulation tools are written in C++, and I will likely be integrating one of these into my tag controller simulator.

I will begin by programming a decoder that can parse the traces correctly and provide the simulator with the data it needs. Next, I will integrate in the cache simulator logic; such cache simulators do exist, and I will likely employ an existing one that has been verified to save unnecessary work, such as DnyamoRIO’s drcachesim [9], allowing me to move forward to the interesting analysis stage which is the heart of the project. Once this is done and tested, I will begin working on analysis tools, which may include visualisations of the tag cache and/or tag table, or statistical properties of the memory accesses. After this, I will implement two existing baseline tag algorithms within the simulator. I intend to implement the hierarchical scheme detailed in *Efficient Tagged Memory*, and also Arm’s Morello tag cache implementation. Evaluating these two is sensible due to my familiarity with both and the proven utility of both. These baselines may serve as a comparison point for a novel algorithm I develop later in the project.

Novel Tag Controller Algorithm (Extension 1)

As an **extension** objective, I aim to develop a novel algorithm for tag table storage. Hopefully the analysis tools will provide useful insight for me to use when developing the algorithm; I intend to use the findings from

the tools to inform the design, and also to investigate other possible techniques, such as the use of a tag cache prefetcher.

I will first need to devise a scheme for dynamically requesting and freeing tag shadowspace DRAM. Once this is done, I will explore how much memory can be saved by a basic algorithm that makes use of the dynamic allocation scheme. After this, I will develop a more advanced algorithm that uses the insights from the analysis tools and my investigation, and I will evaluate the performance of it against those of the two baseline algorithms. I will look at using existing prefetchers if I see them as useful rather than developing my own, as the aim of this extension is to devise a highly performant tag controller algorithm, and not to spend time developing my own prefetcher that likely would not compare against those that have already been optimised and verified.

Hardware Description Language Implementation (Extension 2)

As an **extension** objective, I aim to implement my algorithm in hardware using a hardware description language, check its correctness either by designing a suite of test cases or by formal verification, and evaluate its performance on different workloads.

I would use Bluespec SytemVerilog for this due to the language being designed for hardware implementations just like this and having a suite of tools, including simulators and FPGA synthesis tools, that I can leverage.

FPGA Synthesis (Extension 3)

As an **extension** objective, I aim to synthesise my implementation on an FPGA, check its correctness in a similar fashion, and evaluate its performance on different workloads.

Success Criteria

The core objective is to develop and check the correctness of the tag controller simulator, and to implement and evaluate the two aforementioned existing tag controller algorithms. The project will be successful if this objective is completed. To be specific, I will develop a program that, when given the parameters of the tag table and tag cache and a workload, meets the following criteria:

- it can simulate how the states of the tag table and tag cache change with each memory access of the workload, when employing two different existing algorithms for tag table storage
- for a simulation, it can determine the tag cache's hit ratio
- for a simulation, it can determine the DRAM tag table access frequency
- for a simulation, it can provide at least one other detail about the tag memory accesses that would be useful information to a professional analyst

Plan of Work

1) 16th Oct – 29th Oct

Work Tasks

- Obtain the traces and understand their format and how to parse them.
- Complete development of the trace decoder for the tag controller simulator.

Milestones

- Demonstrate a solid understanding of the format of the trace files by having a working trace decoder that is able to extract all relevant data from a given trace file.

2) 30th Oct – 12th Nov

Work Tasks

- Complete implementation of cache simulator logic.

Milestones

- Demonstrate familiarity with the chosen cache simulator by having it integrated into my tag controller simulator, and verified to be working correctly.

3) 13th Nov – 26th Nov

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II.

4) 27th Nov – 10th Dec

Work Tasks

- Complete the analysis/visualisation tools for the tag controller simulator.

Milestones

- Have checked with my supervisor that the tools would be of use to a professional by having received feedback on a formal document describing what the tools do. Have a finished suite of analysis/visualisation tools that provide useful information to the user.

5) 11th Dec – 24th Dec

Work Tasks

- Implement and evaluate the two existing algorithms using the simulator, and report all findings into a formal document.

Milestones

- Have emailed the document of my findings to my supervisor for review and iterated on any feedback received.

6) 25th Dec – 7th Jan

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II. I will also enjoy the Christmas holidays.

7) 8th Jan – 21st Jan

Work Tasks

- List all opportunities for improvement found, noting which ones I plan to implement in my algorithm and why. Also research techniques that might lead to an improved tag controller algorithm. Detail all my findings in a formal document.
- Make a final list of optimisations I aim to employ in my algorithm and develop a formal specification of my novel algorithm and get feedback from my supervisor.
- Implement the dynamic DRAM allocator into the tag controller simulator.

Milestones

- Have emailed my findings and the final list of optimisations to my supervisor for review, and have iterated on any feedback received.
- Have emailed the formal specification of the algorithm to my supervisor for review.
- Have confirmed with my supervisor that my implementation of the dynamic DRAM allocator does not make any false assumptions about real systems and is an accurate and useful model for the context in which my algorithm may run.

8) 22nd Jan – 4th Feb

Work Tasks

- Implement the algorithm in the tag controller simulator and evaluate its performance, recording all findings in a formal document.
- Write up my progress report, ask for a check from my supervisor if necessary, iterate on any feedback, and submit the final version.
- Prepare my progress report presentation.

Milestones

- Have implemented the algorithm, verified its correctness, evaluated its performance, and written up the formal document of my findings.
- Have completed and submitted my progress report.
- Have completed and rehearsed my progress report presentation to the point of being able to confidently deliver it in 5 minutes.

9) 5th Feb – 18th Feb

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II. I will also deliver my progress report presentation.

10) 19th Oct – 3rd Mar

Work Tasks

- Begin planning the HDL implementation of my algorithm.
- Pick a processor to adjoin the tag controller to, detailing the justification for my choice in a formal document.

- Begin programming the algorithm in my chosen HDL.

Milestones

- Have sent my plan to my supervisor for review and iterated on any feedback.

11) 4th Mar – 17th Mar

Work Tasks

- Finish the HDL implementation and verify that it correctly implements my tag controller algorithm, detailing the verification process in a formal document.
- Begin learning how to go about FPGA synthesis of the HDL implementation.
- Write the introduction and preparation sections of the write-up.

Milestones

- Have sent the formal document of verification details to my supervisor for review and confirmation, and iterated on any feedback.
- Have emailed the introduction and preparation sections to my supervisor and iterated on any feedback.

12) 18th Mar – 31st Mar

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II.

13) 1st Apr – 14th Apr

Work Tasks

- Learn all of the necessary tools/languages/frameworks required to synthesise the HDL implementation onto an FPGA.
- Begin attempting to synthesise the HDL implementation onto the FPGA and working through issues, contacting my supervisor if needed.
- Write the implementation section of the write-up.

Milestones

- Have had a discussion with my supervisor about how exactly to go about the FPGA synthesis.
- Have demonstrated my understanding and ability by synthesising an example netlist onto the FPGA I will use.
- Have emailed the implementation section to my supervisor and iterated on any feedback.

14) 15th Apr – 28th Apr

Work Tasks

- Successfully synthesise the HDL implementation onto an FPGA. Test the FPGA implementation and verify its correctness.

- Evaluate its performance and record all findings into a formal document. Compile this and all previous evaluation and conclusion documents and compose the evaluation and conclusion sections of the write-up.
- Compose the sections into one document that will be the final dissertation. Sent this to my director of studies for feedback, and also to my supervisor if significant changes to previously checked parts have been made.

Milestones

- Have a finalised report of the FPGA’s performance when implementing my tag controller algorithm.
- Have emailed the evaluation and conclusion sections to my supervisor and iterated on any feedback.
- Have received feedback on my entire dissertation from my DoS and supervisor, and have iterated on that feedback. Have submitted the final version of my dissertation.

15) 29th Apr – 10th May

I am leaving this period of time as a planned slack period in which I may catch up with any project work that is behind schedule, and to ensure I am maintaining a good balance with the rest of my work for Part II.

Resource Declaration

Extension 3 will require permission to use a suitable lab-owned FPGA.

Several memory access traces which I intend to use exist on the Computer Lab’s internal network. To use them I will need access to the `/net/archive/export/cheri_traces` directory in which they reside.

My supervisor, Dr Jonathan Woodruff (jdw57), will be responsible for providing both of these resources.

For this project I will be using my laptop which has the following specifications:

- Intel i7 10th Generation Core
- 16 GB RAM
- Dual-boots Windows 10 and Ubuntu 22.04 LTS
- 1.5TB SSD storage

Should this device fail, I will be in need of a new laptop, so I will purchase an at-least equivalently capable machine immediately. I will be using a private GitHub remote repository to provide version control and backups of all my programs, files and documents.

I accept full responsibility for this machine and I have made contingency plans to protect myself against hardware and/or software failure.

References

- [1] Alexandre Joannou et al. “Efficient tagged memory”. In: *2017 IEEE International Conference on Computer Design (ICCD)*. IEEE. 2017, pp. 641–648.
- [2] Brian E. Clark and Michael J. Corrigan. “Application System/400 performance characteristics”. In: *IBM Systems Journal* 28.3 (1989), pp. 407–423.
- [3] Jedidiah R Crandall and Frederic T Chong. “Minos: Control data attack prevention orthogonal to memory model”. In: *37th International Symposium on Microarchitecture (MICRO-37’04)*. IEEE. 2004, pp. 221–232.

- [4] Joseph L Greathouse et al. “A case for unlimited watchpoints”. In: *ACM SIGPLAN Notices* 47.4 (2012), pp. 159–172.
- [5] G Edward Suh et al. “Secure program execution via dynamic information flow tracking”. In: *ACM Sigplan Notices* 39.11 (2004), pp. 85–96.
- [6] Alastair JW Mayer. “The architecture of the Burroughs B5000: 20 years later and still ahead of the times?”. In: *ACM SIGARCH Computer Architecture News* 10.4 (1982), pp. 3–10.
- [7] [https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/hh831766\(v=ws.11\)](https://learn.microsoft.com/en-us/previous-versions/windows/it-pro/windows-server-2012-r2-and-2012/hh831766(v=ws.11)).
- [8] <http://www-ecad.cl.cam.ac.uk/>.
- [9] https://dynamorio.org/page_drcachesim.html.